

VIETNAM NATIONAL UNIVERSITY OF HO CHI MINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



**Estimating Student Demand for Courses Registration Based on
Curriculum Analysis with Dashboard Visualization for Decision
Support**

By **Đinh Vũ Ngọc Linh**

Supervisor: **Dr. Tran Thanh Tung**

*A Thesis submitted to the School of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
Bachelor of Data Science*

Ho Chi Minh City, Vietnam
January, 2026

Estimating Student Demand for Courses Registration Based on Curriculum Analysis with Dashboard Visualization for Decision Support

APPROVED BY:

Committee name here

Committee name here

Committee name here

Committee name here

Committee name here

THESIS COMMITTEE

Acknowledgments

First and foremost, I would like to express my most sincere gratitude to the professors and lecturers of the International University, Vietnam National University Ho Chi Minh City, for imparting invaluable knowledge to me throughout my studies.

In particular, I would like to express my profound gratitude to **Dr. Tran Thanh Tung** for his professional guidance and unwavering support throughout the three months of this thesis. Throughout the three months of completing my graduation thesis, he consistently dedicated his valuable time to providing dedicated guidance, research direction, and answering complex professional questions. His meticulous instruction and insightful advice were key factors in helping me complete this thesis to the best of my ability. This thesis has been an enriching experience that bridged the gap between theory and practice, and I am deeply appreciative of **Dr. Tran Thanh Tung** for making it possible. I am truly grateful for the opportunity to work under his supervision.

Table of Contents

List of Tables	iv
List of Figures	v
List of Algorithms	vi
List of Listings	vii
Abstract	viii
1 INTRODUCTION	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Scope	3
1.3.1 In-Scope: Data Processing and Estimation Logic	3
1.3.2 Out-of-Scope	4
1.4 Objectives	4
2 LITERATURE REVIEW	6
2.1 Current Advances in Course Demand Estimation	6
2.2 Research Gap	7
2.3 Theoretical Background	8
2.3.1 Relational Database Management Systems (RDBMS)	8
2.3.2 Synthetic Data Generation Methodology	10
2.3.3 Advanced SQL Logic and Academic Constraint Modeling	11
3 METHODOLOGY	14
3.1 Database Sources Description	14
3.1.1 Overview of the Database Architecture	14
3.1.2 Data Entities and Relational Logic	14
3.2 Requirement Analysis	17
3.2.1 Problem Statement	17
3.2.2 Stakeholder Requirements	18
3.2.3 Use Case Scenarios	19
3.2.4 Usage Scenarios	19
3.2.5 System Scope	19
3.3 Algorithms and Mathematical Formalization	20
3.3.1 Mathematical Formalization	20
3.3.2 Demand Classification Logic	21
3.3.3 Constraint-Based Simulation Algorithm	22
4 IMPLEMENTATION	24
4.1 Synthetic Data Generation	24
4.1.1 Algorithmic Implementation Logic	24
4.1.2 Code Implementation Snippet	25
4.2 Database Schema Adjustments	27

4.2.1	New Table Implementation: <code>enrollment_batch</code>	27
4.2.2	Modified Table: <code>student</code>	27
4.2.3	Data Integrity Enforcement	28
4.3	Logic Implementation: Demand Estimation Engine	28
4.3.1	Output Structure: The Denormalized Dashboard Table	29
4.3.2	Phase 1: Retake Demand Logic (High Priority)	29
4.3.3	Phase 2: Standard Demand Logic (Curriculum Alignment)	31
4.3.4	Phase 3: Late Demand Logic	32
4.3.5	Phase 4: Fast Demand Logic (Acceleration Strategy)	36
4.4	Dashboard Implementation	38
4.4.1	Data Materialization Strategy	39
4.4.2	Data Visualization Strategy	39
5	RESULT	49
5.1	Experimental Dataset Description	49
5.1.1	Dataset Summary	49
5.1.2	General Dashboard Overview	49
5.1.3	Summary of Experimental Results	60
6	DISCUSSION	61
6.1	Analysis of Results	61
6.1.1	The "Ripple Effect" of Prerequisite Chains	61
6.1.2	Structural Delays: The English Barrier	61
6.1.3	Unveiling Latent Acceleration Demand	62
6.1.4	Simulation Validity: Credit Constraints	62
6.2	Strengths of the Proposed System	62
6.2.1	Precision via Constraint-Based Bottom-Up Forecasting	62
6.2.2	Privacy-Preserving Development Framework	63
6.3	Limitations	63
6.3.1	Synthetic Data Bias vs. Behavioral Nuance	63
6.3.2	Deterministic Rule Sets	63
6.3.3	Supply-Side Absence	64
6.4	Comparison with Traditional Methods	64
7	CONCLUSION	65
7.1	Summary	65
7.2	Future Work	65
7.2.1	Multi-Horizon Forecasting using Machine Learning	65
7.2.2	Real-time Event-Driven Architecture	66
7.2.3	Supply-Side Optimization via Linear Programming	66
7.2.4	Student Advisory Recommender System	67
7.3	Key Contributions	67
7.4	Concluding Remarks	67

List of Tables

4.1	Logic decomposition of the Retake Demand SQL query.	30
4.2	Logic decomposition of the Standard Demand SQL query.	32
4.3	Logic decomposition of the Late Demand Simulation.	35
4.4	Logic decomposition of the Fast Demand Simulation.	37
6.1	Comparison between Manual Estimation and the Proposed System . .	64

List of Figures

2.1	Entity Relationship Diagram (ERD) of the University Database Schema	9
4.1	Distribution of Total Demand by Category (Standard, Late, Fast, Retake).	40
4.2	Top 10 Courses with Highest Projected Enrollment, segmented by Demand Type.	41
4.3	Distribution of Enrollment Demand Flow across Student Batches. . . .	42
4.4	Course Risk Matrix correlating Retake Demand (Y-axis) against Other Demand (X-axis).	43
4.5	Comparative Demand Ranking and Composition Across Batches. . . .	45
4.6	Total Seats Needed (Courses > 50 slots) visualized by Volume and Intensity.	46
4.7	Full Breakdown of Course Demand by Type (Standard, Late, Fast, Retake).	47
5.1	General Dashboard Overview: Key Metrics and High-Level Demand Distribution.	50
5.2	Interactive Scenario: The dashboard filtered by 'Late Demand'. Note the shift in the 'Top 10' chart highlighting backlog bottlenecks.	51
5.3	Cohort Dynamics Dashboard: analyzing demand flow, risk factors, and volume evolution across student batches.	52
5.4	Drill-Down: Dashboard filtered by 'Batch 23' (Juniors). The Ribbon Chart highlights the 'Acceleration Window' (Orange Segment).	53
5.5	Drill-Down: Dashboard filtered by 'Late Demand'. The Sankey lines reveal that backlog issues originate largely from Batches 23 and 24. . .	54
5.6	Infrastructure Planning Dashboard: Granular Demand Matrices and Seat Capacity Treemap.	55
5.7	Interactive Scenario: Selecting 'C Programming' reveals it is a pure Freshman/Sophomore course with minimal irregular demand.	56
5.8	Comprehensive Demand Breakdown (Default View): Showing all courses sorted by total volume with color-coded demand composition.	57
5.9	Interactive Scenario: Filtering by 'Batch 24'	58
5.10	Interactive Scenario: Filtering by 'FAST' demand	59
5.11	Interactive Scenario: Combining filtering by 'FAST' demand and batch 24	59

List of Algorithms

1	Student's Grade simulation	10
2	Retake logic overview	12
3	standard logic overview	12
4	Late logic overview	12
5	Fast logic overview	13
6	Total demand logic overview	13
7	Condition for passing a course	21
8	Eligibility for course enrollment	21
9	Retake Demand Identification	21
10	Standard Demand Identification	21
11	Late Demand (Backlog) Identification	22
12	Fast Demand (Acceleration) Identification	22
13	Priority-Based Course Sorting	22
14	Student Course Basket Simulation	23
15	Final Aggregated Demand Calculation	23
16	Course Selection Queue Assembly	24
17	Stochastic Grade Generation Logic	25
18	Retake Demand Identification Logic	29
19	Standard Demand Identification Logic	31

List of Listings

4.1	Configuration and Difficulty Constants	26
4.2	Capacity and Pace Assignment	26
4.3	Priority Queue and Greedy Allocation	26
4.4	Grade Generation and History Update	27
4.5	Implementation of the enrollment_batch table	27
4.6	Adjusted student table with referential integrity constraints	27
4.7	SQL Implementation for Retake Demand	30
4.8	SQL Implementation for Standard Demand	32
4.9	SQL Simulation for Late Demand (Prerequisite & Credit Check)	35
4.10	Full SQL Implementation for Fast Demand (Acceleration)	37
4.11	Schema Definition for the Dashboard Data Mart	39

Abstract

This thesis addresses the problem of predicting student course enrollment by developing a system for Estimating Student Demand based on Curriculum Analysis. The primary objective is to assist university administrators in making informed decisions regarding course offerings and capacity planning.

Effective study planning is crucial to ensuring that students' learning progress is not hindered and the school's operations are sustained. A key component of this project is estimating course enrollment demand, a process often hampered by the complexity of calculating flexible course allocations based on students' curricula.

The core methodology of this research involves building a data processing procedure using Python and SQL to systematize the number of students who will register for courses in the next semester. By integrating complex logic involving prerequisite chains, parallel course requirements, and semester schedules, the system identifies the group of students eligible for each specific course in the upcoming semester. Unlike simple statistical extrapolation, this method simulates the eligibility status of all enrolled students to generate detailed demand forecasts. This study proposes a bottom-up approach to estimating demand. Instead of relying solely on aggregated historical data, the research focuses on analyzing "supply and demand" within the curriculum itself. The system assesses students' current academic status, considering accumulated credits and completion of prerequisite courses to predict their immediate learning needs. This detailed analysis provides data on the number of students expected to enroll in courses in the next semester, supporting the university in allocating classes efficiently and appropriately to students.

The technical solution is perfected by implementing a dashboard visualization. This serves as a centralized platform for decision-makers, presenting key indicators such as projected student numbers, courses of over-enrollment due to high demand, and the distribution of demand across different subjects. By providing a clear picture of student intentions and conditions, the proposed system allows the university to proactively adjust the courses offered, thereby minimizing enrollment overload, under- or over-enrollment of classes, and supporting students in easily registering for courses they intend to pursue to graduate.

Chapter 1

INTRODUCTION

1.1 Motivation

At my university, International University - VNUHCMC (or shortly is HCMIU), the flexible, credit-based learning program has given students greater autonomy in their studies and time management. However, this flexibility has also created significant operational challenges, particularly in the area of course registration. Course registration has become a frequent source of anxiety and frustration for both students and the university. Each semester, a large number of students face considerable difficulty registering for necessary courses, especially those that are compulsory or crucial prerequisites for graduation, or those that require essential courses. The consequences of this bottleneck are serious: when students miss a crucial prerequisite course due to limited places, it triggers a chain reaction, delaying their ability to take subsequent courses and ultimately prolonging their graduation time.

In stark contrast to the shortage of slots in essential courses, certain elective or less critical courses often witness an overestimation of demand, leading to opened classes with low enrollment. This imbalance represents a significant inefficiency in the utilization of the university's resources, including faculty time, classroom space, and administrative effort. This mismatch between course offerings (supply) and actual student learning needs (demand) suggests a systemic disconnect in the current planning process.

Currently, the process of estimating student demand for course registration largely relies on manual experience, historical heuristics, and rough assumptions. Department administrators often plan the next semester's schedule based on the previous year's data without fully accounting for the dynamic changes in the student body—such as failure rates, changes in majors, or irregular academic progress. This approach lacks a systematic and data-driven foundation, making it inherently reactive rather than proactive. Without granular visibility into the specific academic status of the current student population, it is difficult for the university to make accurate decisions regarding resource allocation, class scheduling, and course planning.

As a Data Science student who has personally experienced the stress of these "registration races," I clearly recognize the urgent need for a more reliable, transparent, and scientific approach to this problem. The motivation for this project stems not only from a desire to solve a personal inconvenience but also from an ambition to apply data analytics techniques to improve university governance.

This thesis is driven by the hypothesis that the data required to solve this problem already exists within the university's records—specifically in student transcripts and curriculum structures. By systematically analyzing the relationships between completed courses and curriculum roadmaps (Curriculum Analysis), it is possible to predict with high accuracy which courses a student becomes eligible for in the next semester. Therefore, the primary goal of this project is to develop a robust estimation method and visualize the findings through a Decision Support Dashboard.

1.2 Problem Statement

Accurately predicting student enrollment demand is a complex and multifaceted challenge facing higher education institutions today, including my university, International University - VNUHCMC. Unlike fixed programs with linear and predictable student flows, the credit system allows for a high degree of flexibility. While beneficial to students, this flexibility introduces a significant element of randomness into the planning process, making demand estimation difficult.

Currently, decisions regarding the number of places offered each semester are primarily based on manual estimates and empirical assumptions from faculty and departments. This approach often relies on past enrollment data—simply looking at how many students enrolled in the previous year—without fully accounting for the dynamic changes in current student numbers. This method lacks a systematic, data-driven foundation and fails to address several key complexities:

1. **Complex Prerequisite Dependencies:** The most significant barrier to accurate estimation is the complex network of prerequisites within the curriculum. Student participation in a course is tightly regulated by a chain of prerequisites (e.g., students must pass Course A to take Course B). Manual estimation methods often overlook accurately calculating the actual number of students who have passed Course A and are ready for Course B. Furthermore, when a large number of students fail a prerequisite course, demand for the next course drops sharply, while demand for retaking the failed course skyrockets. Manually tracking these "spillover effects" across thousands of students and hundreds of courses is nearly impossible.

2. **Heterogeneity of Student Progress:** Students do not progress at the same pace. The student community includes full-time students following the standard curriculum, students retaking courses to improve their grades, students falling behind due to completing English placement tests, and students deferring or taking a semester off. There is also a segment of "extracurricular" students who may learn faster or slower than their group. Simply counting the number of students currently enrolled does not reflect true demand, as a significant portion may not qualify for or be interested in specific courses in the upcoming semester.

3. **Resource Imbalance and Consequences:** The inaccuracies in current estimation methods lead to a serious imbalance between the number of courses offered (supply) and the actual student demand. In fact, some elective or specialized courses that attract little student interest are over-allocated, resulting in wasted teaching resources and empty classrooms. Conversely, core courses in high demand frequently face a shortage of places. This forces students to compete for spots or postpone essential courses to later semesters. These delays disrupt individual learning plans, create bottlenecks in the curriculum flow, and significantly increase the risk of students falling behind their expected graduation timeline.

4. **Data Fragmentation:** The data needed to address this problem—student transcripts, curriculum rules—often exist individually or are not processed in a way that supports predictive modeling. There is a lack of an integrated tool that can aggregate this data to provide "big picture" of student needs.

Therefore, this project aims to address these multifaceted problems by abandoning intuitive planning methods. By calculating course needs through rigorous analysis of curriculum structure combined with past course registration data, this thesis seeks to develop a more accurate, automated method for estimating student needs. This will serve as a foundation for more effective learning planning and resource optimization.

1.3 Scope

The main objective of this thesis is to build a robust, data-driven system capable of predicting student course registration needs. By leveraging past academic data and digitizing curriculum regulations, this project aims to provide an alternative to outdated and ineffective methods, enabling more accurate estimation of actual student enrollment demand. The scope of the research is defined as follows:

1.3.1 In-Scope: Data Processing and Estimation Logic

The core of the project revolves around an algorithmic approach to model student registration behavior. The scope clearly includes the following analytical components:

- **Mapping Curriculum Dependencies:** The system digitizes a complex network of course registration prerequisites spanning different academic curriculum across different student cohorts. This includes not only mapping standard semester pathways but also rigorously enforcing subject-to-subject relationships (e.g., Prerequisite course) to ensure that anticipated needs comply with academic regulations.
- **Comprehensive Needs Segmentation:** Unlike traditional methods that treat all students equally, this project categorizes enrollment needs into four distinct behavioral groups based on individual academic history:
 - *Retake Demands:* Identify students who need to retake courses they previously failed in order to meet prerequisite requirements and to meet graduation requirements.
 - *Standard Demands:* Estimate the number of students who will catch up on the recommended curriculum for their batch.
 - *Late Demands:* Identify students who need to register for courses from previous semesters that they missed, falling behind in their studies due to various factors (e.g., taking a year off, attending intensive English courses, or being unable to register due to scheduling conflicts, etc.).
 - *Fast Demands:* Estimates the number of high-achieving students eligible to register for courses in a future semester, courses in the following semester after the standard semester of the curriculum.
- **Constraint-Based Registration Simulation:** A key feature of this scope is the implementation of "Credit Limit Simulation". The algorithm doesn't simply add up all eligible courses; instead, it prioritizes courses (e.g., retake and late courses first, then standard courses) and stops calculating needs when students reach the maximum number of credits allowed per semester that is 24 credits. This ensures the forecast figures are realistic and actionable.
- **Eligibility Filtering:** This scope includes processing conditional logic where appropriate, such as filtering students based on specific prerequisites (e.g., English proficiency results) before placing them into the needs group.

- **Visualization and Reporting:** The technical product includes the development of a interactive dashboard. This interface processes estimated data to visualize key indicators, such as the total number of slots needed for each course and the allocation of demand among student cohorts for different courses, facilitating immediate decision-making by academic administrators.

1.3.2 Out-of-Scope

To ensure the research focuses on quantitative needs estimation, the following aspects are excluded:

- **Qualitative Preference Modeling:** This system assumes students act rationally to complete their degree. It does not account for subjective preferences influencing enrollment, such as the popularity of a particular faculty member, the desire for "easy" elective courses, or group enrollment behavior.
- **Logistical Timetabling:** This project focuses on the "*What*" and "*How many*" (identifying courses and number of students) rather than "*When*" (scheduling). Issues related to overlapping schedules, or faculty availability are outside the scope of this needs estimation model.
- **Administrative Exclusion Factors:** External administrative factors that could prevent students from enrolling, such as tuition debt or suspension due to disciplinary violations, are not integrated into the estimation logic due to privacy and data availability constraints.
- **Unforeseeable Learning Interruptions:** The demand prediction model is based on the assumption of continuous learning. It does not account for unforeseeable interruptions such as students suddenly postponing results, dropping out mid-semester, or transferring to another university.

In summary, this thesis operates within the field of Educational Data Mining, focusing strictly on the computation of student admission eligibility logic and demand simulation. By transforming curriculum rules and student transcripts into structured needs indicators, the project provides a foundational tool for optimizing teaching planning, independent of subjective or logistical external factors.

1.4 Objectives

The overarching objective of this thesis is to design a robust, automated Decision Support System specifically tailored for university academic planning. This research aims to replace reliance on manual methods and intuitive estimation with a data-driven curriculum analysis framework. By integrating student academic history with digitized curriculum logic, the proposed system seeks to accurately predict course enrollment demand, thereby enabling the university to optimize resource allocation, minimize scheduling bottlenecks, and facilitate timely student graduation.

To achieve the overall goal, this project focuses on accomplishing the following specific technical and analytical objectives:

1. To Digitize and Model Curriculum Logic:

- Building a relational database schema that fully captures the complexity of credit-based curricula, including directed graph relationships between courses (prerequisites, previous, and Co-requisite). Mapping standard learning paths among different student groups (enrollment periods) to establish a basis for "ideal" student progress.

2. Developing a multi-segment demand estimation algorithm:

- Designing and implementing SQL-based procedures capable of classifying student demand into four different behavioral categories:
 - (a) *Retake demand*: Identify students who have failed specific courses and need to re-register immediately.
 - (b) *Standard demand*: Estimate the number of students who will naturally progress to the assigned courses in the next semester based on their curriculum.
 - (c) *Late demand*: Detect students who have fallen behind and need to take backlog courses from previous semesters.
 - (d) *Fast demand*: Estimates the number of high-achieving, fast-tracking students eligible to pre-register for future courses.

3. To Implement Constraint-Based Registration Simulation:

- Goes beyond simple counting by integrating a **simulation tool** that mimics the actual student registration constraints.
- Integrates logic that rigorously enforces prerequisite checks (ensuring only eligible students are counted) and applies a **maximum credit threshold** (e.g., simulating that a student will stop registering when they reach 24 credits). This ensures the predicted demand reflects actual capacity, not theoretical intent.

4. To Design an Interactive Decision Support Dashboard:

- To develop a dynamic visualization Dashboard that aggregates the processed data into actionable insights.
- Provide instructors with the ability to view demand indicators at a detailed level, such as demand for specific courses, and allocate demand by student group, enabling data-driven decisions for place allocation.

To demonstrate that the algorithmic approach provides a more precise reflection of student needs compared to traditional manual estimation methods.

Chapter 2

LITERATURE REVIEW

2.1 Current Advances in Course Demand Estimation

The field of academic planning and course demand forecasting has seen a significant shift in recent years, from traditional empirical methods to advanced data-driven approaches. This development is largely driven by the rapid growth of Educational Data Mining (EDM) and Learning Analytics (LA), which aim to extract useful insights from vast educational data repositories. Current advances in this field can be categorized into three main streams: historical trend analysis, machine learning-based prediction, and curriculum-dependent simulation.

Traditionally, university administrators relied on "forward" methods, where courses offered for the upcoming semester were based on enrollment figures from the same semester of the previous year, slightly adjusted. However, recent literature highlights the limitations of this approach, noting that it cannot account for curriculum changes or student anomalies. Modern advancements have seen the adoption of centralized data warehouses in higher education. Organizations are increasingly moving from disparate Excel spreadsheets to integrated databases, consolidating Student Information Systems (SIS) and Learning Management Systems (LMS). This infrastructure enables real-time querying of student learning status, serving as the foundational layer for any advanced needs estimation model.

A prominent trend in current research concerns the application of machine learning (ML) algorithms to predict student enrollment behavior. Many studies use models such as ARIMA (Auto-Regressive Integrated Moving Average) or LSTM (Long Short-Term Memory) networks to analyze past course registration logs. These models treat course registration as a time-dependent variable, predicting future demand based on seasonality and past trends. Other researchers approach the problem at the student-by-student level, using classification algorithms such as Random Forests, Decision Trees, or Logistic Regression to predict the probability that a particular student will choose a particular elective course based on their past scores and preferences.

While these machine learning methods show promise in predicting trends in elective courses or open enrollment, they are often ineffective in rigorous credit systems. Purely statistical models may fail to capture hard constraints – such as prerequisite chains or credit limits. For example, a machine learning model might predict a high probability that a student will take the "Algorithms and Data Structure" course based on interest, ignoring the fact that the student has not passed the required "Object-Oriented Programming" prerequisite. This gap highlights the need for constraint-based or rule-based methods.

Recognizing the limitations of purely statistical methods as well as machine learning models, a recent wave of progress has focused on "Curriculum Analysis." This approach views the curriculum as a complex network or directed graph. Researchers are applying Process Mining techniques to explore the actual paths students take versus the ideal paths designed by instructors. Current advances in this area aim to formalize cur-

riculum rules (prerequisites, concurrent conditions) into logic layers that dynamically verify student eligibility. This ensures that demand forecasts are not only highly valid but also academically valuable.

The final part of current advances lies in the presentation layer. Generating accurate data is insufficient without effective communication tools. There is a growing trend toward integrating Business Intelligence (BI) tools (such as Power BI, Tableau, or custom web-based dashboards) into academic governance. Modern DSS in education goes beyond static reporting; it provides interactive capabilities that allow administrators to delve deep from university-level data to faculty and course-level metrics, thereby providing invaluable support to administrators.

2.2 Research Gap

Despite the extensive literature on Educational Data Mining (EDM) and enrollment forecasting, there remains a critical disconnect between theoretical prediction models and the practical, rule-bound reality of university course registration. While existing studies have successfully applied Machine Learning techniques to predict student performance [9, 7] or recommend elective courses based on interest [17], several significant research gaps persist when applied to a strict credit-based curriculum environment:

Most current advancements rely heavily on probabilistic Machine Learning models (e.g., Collaborative Filtering or Regression) to predict course demand. Specifically, studies such as Elbadrawy et al. [17] and Polyzou et al. [19] utilize matrix factorization and specific models to suggest courses. While effective for identifying student interests, these models often fail to account for the **deterministic nature** of academic regulations. A pure ML model might predict a high probability of a student enrolling in a course based on latent features, ignoring the binary constraint of prerequisite failures. Existing literature often treats curriculum rules as weak features rather than hard constraints, leading to forecasts that are statistically plausible but academically invalid [10].

Traditional forecasting methods typically produce aggregate numbers or focus on individual grade prediction [7]. However, they rarely distinguish the *source* of this demand. There is a scarcity of research that segments demand into behavioral categories such as **Retake**, **Standard**, or **Late/Fast**. Most existing systems treat all demand units as homogeneous, whereas distinct student groups require different resource allocation strategies.

Perhaps the most significant gap is the lack of **credit-limit simulation** in demand estimation. Most course recommendation frameworks assume "infinite capacity" or suggest a "Top-N" list of courses [17], summing up all courses a student *could* potentially take. In reality, students are bounded by maximum credit limits (e.g., 24 credits/semester). Existing literature often overlooks this prioritization process (Logic: Retake $\rightarrow$ Standard $\rightarrow$ Elective), resulting in consistently over-estimated demand figures that do not reflect the actual registered credits.

Finally, while many studies focus on the algorithmic complexity of prediction models [9], fewer studies address the translation of these outputs into accessible tools for academic administrators [11]. There is a lack of integrated frameworks that combine rigorous back-end curriculum analysis with front-end **Dashboard Visualization** designed specifically for slot allocation. The gap lies in bridging the algorithmic findings with operational management needs.

This thesis aims to fill these gaps by moving away from purely probabilistic predictions. Instead, it proposes a **deterministic, rule-based simulation approach** that strictly enforces prerequisite chains, segments demand types, and respects credit limits, visualized through a decision support dashboard.

2.3 Theoretical Background

This part provides a comprehensive overview of the theoretical underpinnings and technical methodologies that govern the architecture of the proposed Course Demand Estimation System. The discussion serves as a foundation for understanding how raw academic data is transformed into actionable insights. The chapter is structured around three critical pillars:

1. **Relational Database Management:** exploring the principles of data normalization and entity-relationship modeling required to maintain the integrity and consistency of complex academic records.
2. **Statistical Methods for Data Synthesis:** detailing the mathematical techniques and probability distributions utilized to generate robust synthetic datasets, ensuring the system’s reliability even in the absence of massive historical logs.
3. **Advanced SQL Logic:** elucidating the procedural programming techniques—including Stored Procedures, Window Functions, and CTEs—employed to translate intricate curriculum regulations into executable analytical algorithms.

2.3.1 Relational Database Management Systems (RDBMS)

The foundation of the Course Demand Estimation System is built upon a Relational Database Management System (RDBMS). An RDBMS allows for the modeling of complex real-world entities and their interrelationships through a tabular schema. This project utilizes the relational model to map the intricate structure of the university’s curriculum, transforming abstract academic regulations into structured, queryable data.

MySQL was chosen as the implementation platform due to its high performance, scalability, and robust support for complex querying logic. The system implementation leverages several advanced RDBMS features to handle the specific challenges of student demand estimation: In the context of an academic management system, the preservation of **Data Integrity** is paramount. RDBMS ensures this through two primary mechanisms:

- **Entity Integrity:** Enforced via Primary Keys, ensuring that every student, course, or program has a unique identifier.
- **Referential Integrity:** Enforced via Foreign Keys, ensuring that relationships between tables remain consistent. For example, a **Result** record cannot exist unless it references a valid **Student_ID** and **Course_ID**.

Furthermore, RDBMS supports the **ACID** properties (Atomicity, Consistency, Isolation, Durability), which are critical for transactional reliability. This ensures that complex operations—such as calculating the demand for thousands of students simultaneously—are processed accurately without data corruption, even in the event of a system failure.

Implementation in Course Demand System

For this project, **MySQL** is selected as the core database engine. MySQL is an open-source, relational database that employs the **InnoDB** storage engine to support standard SQL transactions and foreign key constraints. The system implementation leverages several advanced features to handle the specific challenges of student demand estimation:

1. **Complex Schema Modeling:** The database schema acts as a digital twin of the university’s academic structure. It includes core entities such as **Student**, **Course**, and **Program**, but also extends to relational tables like **enrollment_batch** (mapping students to their curriculum year) and **course_course_relationship** (defining prerequisite graphs). This structure allows for multi-dimensional analysis, linking a student’s past performance directly to their future eligibility.

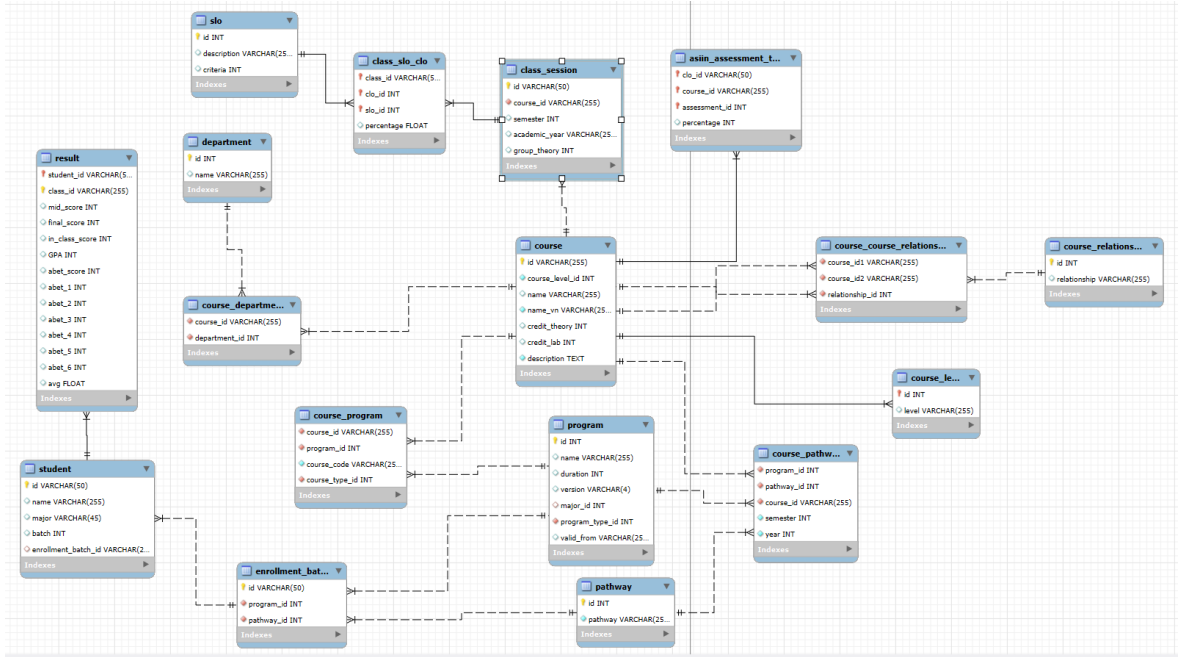


Figure 2.1: Entity Relationship Diagram (ERD) of the University Database Schema

2. **Data Consistency and Integrity:** Given the sensitivity of academic records, the system relies on the RDBMS’s strict enforcement of constraints. Foreign keys play a crucial role in maintaining the validity of the estimation data; for instance, ensuring that a "Retake Demand" is only generated for students who genuinely exist and have a valid grade history.
3. **Server-Side Processing:** A distinctive feature of this implementation is the utilization of **MySQL Stored Procedures** to handle heavy computational logic. Instead of fetching raw data to an external application, the demand estimation algorithms (classifying students into 'Standard', 'Retake', or 'Fast' groups) are executed directly within the RDBMS engine. This approach minimizes data latency and maximizes the efficiency of the simulation process.

2.3.2 Synthetic Data Generation Methodology

In the domain of Educational Data Mining, acquiring high-quality datasets is often hindered by strict data privacy regulations (such as GDPR or internal university policies) designed to protect Personally Identifiable Information (PII). Furthermore, historical data may be sparse or unbalanced, limiting the ability to test the system’s robustness under extreme scenarios. To overcome these limitations, this research adopts a **Synthetic Data Generation** approach.

This methodology involves the algorithmic creation of artificial datasets that preserve the *statistical fidelity* and *structural logic* of real-world academic records without containing any sensitive information. The generation pipeline is designed not merely to produce random noise, but to construct a coherent “Academic Digital Twin” of the university environment.

Probabilistic Modeling of Student Capacity

To simulate realistic grade outcomes, the system introduces a latent variable denoted as S_{cap} (Student Capacity), representing an individual student’s academic aptitude. This variable is not uniform but is modeled using specific probability distributions to reflect the diversity of the student body:

- **Normal Distribution (Gaussian) for Active Cohorts:** For standard cohorts (e.g., batches K21-K24), academic capacity is assumed to follow a Bell Curve, $X \sim \mathcal{N}(\mu, \sigma^2)$. This assumption reflects the pedagogical reality where the majority of students possess average capability, while high-performers (Outliers on the right) and struggling students (Outliers on the left) are less frequent.
- **Skewed Distribution for Delayed Cohorts:** For older cohorts (e.g., K17-K20) who have exceeded the standard 4-year curriculum duration, the capacity model is adjusted to a **Left-Skewed Distribution**. This reflects a “survivorship bias” in reverse: students remaining in the system after the standard graduation time typically have a history of lower performance, higher failure rates, and accumulated course debts.

The grade G for a specific student i in course j is then generated as a function of their capacity $S_{cap}^{(i)}$ and the course difficulty D_j , with an added stochastic noise term ϵ to account for randomness:

Algorithm 1 Student’s Grade simulation

$$G_{ij} = f(S_{cap}^{(i)}, D_j) + \epsilon \quad (2.1)$$

Sequential Discrete Event Simulation (DES)

Rather than generating static, unconnected data points, the system employs a **Discrete Event Simulation** framework to model the temporal progression of studies. The generation process iterates through a chronological timeline, semester by semester, from the student’s admission date to the current timestamp. This approach ensures:

- **State Dependency:** A student’s eligibility for a course in Semester T is strictly dependent on the results generated in Semester $T - 1$. For instance, the system

will not generate a grade for “Data Structures” if the student failed the generated attempt for “Intro to Programming” in the previous simulated semester.

- **Accumulation of Academic History:** As the simulation progresses, the system builds a cumulative GPA and a history of passed/failed courses. This allows for the realistic generation of “Retake” scenarios, where a student must repeat a course in a subsequent semester, thereby populating the “Retake Demand” segment in the database.

Structural Integrity and Validation

Finally, the synthetic data is subjected to rigorous constraint checks to ensure it adheres to the relational schema defined in the RDBMS.

- **Curriculum Alignment:** Generated enrollments are restricted to courses that actually exist within the program structure of the student’s major.
- **Logical Constraints:** The system enforces logical rules, such as preventing a student from taking more than the maximum allowed credits in a single simulated semester, or ensuring that a student cannot graduate without completing all compulsory credits.

2.3.3 Advanced SQL Logic and Academic Constraint Modeling

The core intelligence of the Course Demand Estimation System resides in its database layer. Unlike simple transactional systems that merely record data, this system acts as a computational engine using SQL. To ensure the generated demand forecasts are actionable and realistic, the system implements a rigorous constraint satisfaction model based on Graph Theory and Set Theory.

Prerequisite Logic as Directed Acyclic Graphs (DAG)

In a credit-based curriculum, courses cannot be treated as independent entities. They are nodes in a complex dependency network.

- **Graph Representation:** The curriculum is modeled as a Directed Acyclic Graph $G = (V, E)$, where V represents the set of courses and E represents the prerequisite relationships. An edge directed from Course A to Course B ($A \rightarrow B$) implies that a student must successfully complete A before attempting B .
- **Traversal Algorithm:** To determine eligibility, the SQL logic performs a graph traversal check. For any target course C_{target} , the system verifies the “visited” status of all parent nodes. This is implemented using NOT EXISTS subqueries to filter out students who have not yet passed the required prerequisite subjects, ensuring that the predicted demand does not violate academic regulations.

Constraint-Based Simulation via Window Functions

A critical innovation in this system is the application of **Credit Constraint Simulation**. Simply counting all eligible courses often leads to inflated demand figures (e.g., a student eligible for 40 credits but only allowed to take 24). To solve this, the

system employs SQL **Window Functions** (specifically `SUM() OVER (PARTITION BY ... ORDER BY ...)`) to simulate the registration process:

1. **Prioritization:** Courses are ranked by priority: *Retake* (Highest) $\rightarrow$ *Late* $\rightarrow$ *Standard* $\rightarrow$ *Fast* (Lowest).
2. **Course Relationship Constraint :** The system use a filter running the course relationship constraint (Prerequisite, Co-requisite, Previous) for each student.
3. **Cut-off Threshold:** The algorithm applies a filter (e.g., `Accumulated_Credits <= 24`), effectively discarding lower-priority courses once the student's credit basket is full.

Demand Segmentation Techniques

The system utilizes advanced Set Theory operations within SQL to classify student demand into four distinct behavioral categories:

- **Retake Demand:** This logic identifies academic failures that require remediation. It is defined as the set of courses where a student has attempted but not yet passed, excluding any courses currently in progress.

Algorithm 2 Retake logic overview

$$Demand_{retake} = \{c \in C \mid grade(s, c) < 50\} \setminus \{c \in C \mid grade(s, c) \geq 50\} \quad (2.2)$$

In this equation, $grade(s, c) < 50$ identifies the set of failed attempts for student s in course c , while the subtraction ensures that any course eventually passed in a later attempt is removed from the retake list.

- **Standard Demand:** This predicts the “natural” progression of students by mapping a student's `enrollment_batch` (e.g., K21) to the curriculum's temporal timeline. It identifies the set of courses mandated for the target semester t .

Algorithm 3 standard logic overview

$$Demand_{standard} = \{c \in C \mid semester(c, Program_s) = t\} \quad (2.3)$$

The function $semester(c, Program_s)$ returns the recommended semester index for course c within the student's specific program roadmap.

- **Late Demand:** This segment detects “debt” courses—subjects from previous semesters that the student has skipped or missed. The logic scans the historical roadmap and identifies gaps in the student's academic record.

Algorithm 4 Late logic overview

$$Demand_{late} = \{c \in C \mid semester(c, Program_s) < t\} \setminus \{c \in C \mid \exists grade(s, c)\} \quad (2.4)$$

$\{c \mid \text{semester} < t\}$ represents all courses that should have been taken in the past, and $\exists \text{grade}(s, c)$ filters out any course that already has an existing grade (regardless of pass/fail) in the system.

- **Fast Demand:** For high-performing students, the system scans future semesters to identify eligible courses for acceleration. This requires validation of prerequisite chains.

Algorithm 5 Fast logic overview

$$Demand_{fast} = \{c \in C \mid \text{semester}(c, Program_s) > t \wedge \text{is_eligible}(s, c)\} \quad (2.5)$$

The predicate $\text{is_eligible}(s, c)$ confirms that the student has completed all prerequisite courses and meets the GPA requirements to enroll in advanced subjects.

- **Total Demand:** The aggregate demand for a specific student in the target semester is the union of all identified segments, ensuring no duplication of course codes.

Algorithm 6 Total demand logic overview

$$Demand_{total} = Demand_{retake} \cup Demand_{standard} \cup Demand_{late} \cup Demand_{fast} \quad (2.6)$$

Procedural Encapsulation

Finally, all the aforementioned logic is encapsulated within MySQL **Stored Procedures**. This design choice offers several advantages:

- **Atomic Execution:** The demand calculation involves multiple steps (clearing old data, calculating new demand layers). Stored Procedures ensure these steps are executed as a single atomic transaction.
- **Performance Optimization:** By processing data on the server side, the system minimizes network overhead and leverages the database engine's internal caching mechanisms for faster execution.

Chapter 3

METHODOLOGY

3.1 Database Sources Description

This project adopts a systematic, data-driven approach to estimate student demand for course registration. Unlike probabilistic machine learning models that predict outcomes based on statistical likelihoods, this methodology employs a deterministic simulation engine that relies on the rigorous analysis of structured historical data, curriculum constraints, and academic regulations to produce demand forecasts that are not only accurate but also fully explainable and aligned with the university’s operational context.

3.1.1 Overview of the Database Architecture

The foundation of the demand estimation system is a robust relational database, specifically designed to digitize the university’s curriculum management. This database serves as the “Single Source of Truth” for all academic structures, ensuring consistency, transparency, and traceability across the estimation process.

The schema is normalized to effectively manage complex relationships between academic entities. It acts as a digital backbone containing:

- **Structural Data:** Defining the static hierarchy of programs, majors, and course catalogs.
- **Logic Data:** Encodes the rules for course registration, such as networks of prerequisite courses and co-requisite requirements.
- **Temporal Data:** Mapping courses to specific semesters within a 4-year roadmap, which is crucial for predicting standard student progression.

3.1.2 Data Entities and Relational Logic

To facilitate precise demand estimation, the project extracts and utilizes data from several critical tables. These entities are categorized based on their role in the simulation logic:

Curriculum Structure Entities

These tables define the fundamental units of the academic curriculum.

course: This table functions as the master dictionary for all subjects. Key attributes include `id` (Primary Key), `course_level_id` - show the course belong to which program (Undergraduate or Master), `name` course name in English, `name_vn` course name in Vietnamese, `credit_theory`, `credit_lab` - credit of the course (theory and lab), and a detailed `description`.

program: This table serves as the central registry for all academic degree programs. Key attributes include **id** (Primary Key), **name** for the official title of the program, and **duration** which specifies the expected time-to-completion (e.g., in years or semesters). To support curriculum evolution, the **version** attribute tracks different iterations of the same program, while **valid_from** indicates the effective date of a specific version. Furthermore, it includes **major_id** to link the program to a specific academic field and **program_type_id** to categorize the degree level (e.g., Bachelor, or Engineering degree).

course_program: This associative table defines the specific inclusion of subjects within an academic program, bridging the gap between the master course list and degree-specific requirements. Key attributes include **course_id** and **program_id**, which establish the many-to-many relationship between courses and programs. The **course_code** attribute stores the unique identifier for the subject as it is recognized within that specific program (e.g., "CS101"). Furthermore, the **course_type_id** categorizes the subject's role within the curriculum, such as whether it is a General Education requirement, a Core Major subject, or an Elective. This table is essential for ensuring that the same course can be shared across multiple programs while maintaining unique codes and classifications for each.

pathway: This table serves as a reference for the various specialized tracks or curriculum variations available within a major. It contains a unique **id** (Primary Key) and a **pathway** attribute, which stores the descriptive label of the academic route (e.g., "Standard," "Honors," or "Fast-track"). This entity is essential for supporting programs that offer multiple learning paths, ensuring that the system can distinguish between different sets of graduation requirements even when students belong to the same program and major.

Constraint and Logic Entities

These tables define the rules that restrict student registration, forming the core of the validation logic.

course_relationship: This lookup table acts as a dictionary for the various types of academic dependencies that can exist between subjects. It consists of a unique **id** (Primary Key) and a **relationship** attribute which defines the nature of the constraint—for example, "Prerequisite" (must be completed before), "Corequisite" (must be taken at the same time), or "Parallel" (must be taken before but not necessarily passed). This categorization is fundamental to the system's logic engine, as it dictates how the stored procedure validates a student's eligibility to enroll in "Late" or "Fast" track courses based on their academic history.

course_course_relationship: This table functions as the enforcement layer for academic dependencies by mapping binary relationships between specific subjects. Key attributes include **course_id1** (the target subject) and **course_id2** (the required or related subject), while **relationship_id** specifies the constraint type (e.g., prerequisite). By storing these pairings, the system can programmatically ensure that students are not counted toward the demand for advanced courses—particularly in "Fast-track" or "Standard" scenarios—unless they have

successfully cleared the necessary prior requirements. This table is the core technical component used in the stored procedure's prohibitive logic to maintain curriculum integrity and realistic planning.

Temporal Planning Entities (The "Roadmap")

These tables are used to calculate **Standard Demand**.

course_pathway: This table acts as the definitive academic roadmap, establishing the ideal timeline for course completion within each program and specific track. Key attributes include **program_id**, **pathway_id**, and **course_id**, which associate subjects with their corresponding degree tracks. Crucially, the **year** and **semester** attributes specify the recommended sequence of study. This temporal data serves as the foundation for the system's logic to calculate **Standard Demand** and is used as a benchmark to identify **Late** demand (backlogs) or **Fast** track opportunities by comparing a student's actual progress against this theoretical schedule.

enrollment_batch: This table functions as the connective tissue between student cohorts and their designated academic trajectories. Key attributes include **id** (Primary Key), which identifies a specific intake group (e.g., Batch 2021), and the foreign keys **program_id** and **pathway_id**. By anchoring a batch to a specific program and pathway, this entity allows the system to determine exactly which curriculum roadmap applies to which group of students. In the demand calculation logic, this table is essential for joining student records with the **course_pathway**, enabling the stored procedure to calculate the expected "Standard" progress based on the batch's seniority.

Transactional Data (The "History")

These tables provide the dynamic input required to calculate **Retake** and **Late Demand**.

student: Stores profile information of all students. Key attributes include **id** (Primary Key), **name**, and **major**, which identifies the student's field of specialization. The **batch** attribute is particularly significant for temporal logic, as it represents the student's intake year (e.g., 21 for 2021), allowing the system to calculate their current academic seniority relative to a target semester. Additionally, the **enrollment_batch_id** acts as a foreign key that links each student to a specific program and curriculum pathway. In the demand forecasting process, this table provides the base population; the stored procedure iterates through these records to determine if a student is on track, falling behind, or eligible for acceleration.

result: This table serves as the comprehensive repository for academic performance history, capturing the outcome of every course attempt by a student. Key attributes include **student_id** and **class_id**, which link a student's grade to a specific course offering. The table stores granular performance data including **mid_score**, **final_score**, and **in_class_score**. Most critically, the **avg** (average) attribute is the primary driver for the system's dynamic logic; it is used to identify **Retake** demand (when **avg**<50) and to verify if a student has satisfied the prerequisites stored in the constraint entities. Additionally, the table includes **GPA** and **abet_score** (along with specific ABET criteria from **abet_1** to **abet_6**) to support accreditation tracking and overall academic standing.

class_session: This table acts as the bridge between the theoretical curriculum and actual academic delivery, representing specific instances of a course being offered. Key attributes include **id** (Primary Key) and **course_id**, which identifies the subject taught. The temporal context is defined by **semester** and **academic_year**, recording exactly when the class was held. Additionally, the **group_theory** attribute distinguishes between different sections or groups of students taking the same course in the same term. In the demand calculation logic, this table is indispensable for resolving academic history; it allows the system to link a student's **result** back to a specific course, which is essential for identifying whether a student has passed, failed (triggering **Retake** demand), or skipped a mandatory subject (triggering **Late** demand).

3.2 Requirement Analysis

The initiative to develop the Course Demand Estimation System is driven by the imperative to optimize academic planning and resource allocation at International University- VNU-HCMC (IU). Currently, the estimation of course slots relies heavily on manual heuristics or semi-automated processes based on historical roll-forward data. These traditional methods often result in operational inefficiencies, such as the under-provisioning of popular courses leading to registration bottlenecks, or the over-provisioning of low-demand electives resulting in wasted faculty resources.

3.2.1 Problem Statement

The Academic Affairs Office faces significant, multifaceted challenges in accurately predicting the enrollment volume for specific courses in upcoming semesters. These challenges stem from three primary root causes:

1. **Stochastic Student Progression:** Unlike fixed-cohort systems, the credit-based curriculum allows students to progress at variable rates. Factors such as course failures, retakes, gap semesters, or accelerated study plans (fast-tracking) create a heterogeneous student body. Consequently, a student's enrollment year (e.g., K21) is no longer a definitive indicator of their current academic standing.
2. **Tightly coupled dependency relationships in the curriculum:** The curriculum is structured as a complex directed acyclic graph (DAG) of prerequisites. Whether students are eligible to enroll in an advanced course depends entirely on their success in prior courses. Manual estimation methods cannot verify these dependency relationships at scale due to the complexity of the computations and rules applied to massive datasets, leading to "biased" demand forecasts (including students who wish to take a course but are not yet eligible).
3. **Data fragmentation and latency:** Critical data points—such as student academic outcomes, curriculum pathways, and English proficiency levels—are scattered across normalized database tables. Extracting actionable insights requires complex joins and aggregations, which are very difficult to perform manually or through standard spreadsheets.

3.2.2 Stakeholder Requirements

Based on the operational needs of the academic planners and administrators, the system is designed to fulfill the following comprehensive requirements:

Functional Requirements (FR)

The system must provide specific analytical capabilities to support decision-making:

- **FR-01: Multi-segment demand classification:** The system must classify student demand into four distinct behavioral groups to enable targeted resource allocation:
 - *Retake demand (Priority 1):* Identify students who have failed a mandatory course and need to re-enroll immediately to stay on track for graduation.
 - *Standard demand (Priority 2):* Estimate the number of students who are naturally progressing on schedule according to the standard curriculum pathway for their respective cohorts.
 - *Delayed/backlog demand (Priority 3):* Detect students who missed or skipped courses in previous semesters (due to schedule conflicts, a gap year for personal reasons, or being held back in English classes and thus studying later) and who now need to complete these outstanding courses.
 - *Fast-track/accelerated demand (Priority 4):* Identify high-performing students who have completed prerequisite courses early and are eligible to enroll in courses ahead of the standard schedule.
- **FR-02: Enforcement of course prerequisite constraints:** The estimation logic must strictly comply with academic regulations. The system must verify prerequisite course chains (both direct and indirect) and exclude any student from the demand set of a target course if they have not satisfied the requirements of that target course (mandatory pass, mandatory prior enrollment, etc.).
- **FR-03: Maximum credit limit simulation:** To provide realistic demand figures, the system must comply with enrollment constraints on the maximum number of credits per semester as stipulated by the university (maximum 24 credits at IU). The algorithm should prioritize courses (Retake → Late → Standard → Fast) and stop counting additional demand for a student once their simulated registration basket is full.
- **FR-04: Detailed cohort analysis:** The system must provide in-depth analysis capabilities, allowing administrators to view demand segmented by specific admission cohorts (e.g., K21, K22, K23) and by individual courses to identify bottleneck trends for each cohort and address issues related to specific courses.

Non-Functional Requirements (NFR)

- **NFR-01: Data privacy and compliance:** The system must be developed and tested without exposing the Personally Identifiable Information (PII) of real students; instead, it will use detailed, simulated synthetic data. This requires the implementation of a statistical engine to generate a high-fidelity synthetic dataset that reflects the characteristics of real academic records.

- **NFR-02: Scalability and Performance:** The database architecture and SQL procedures must be optimized to handle the processing of thousands of student records and millions of result entries with minimal latency.
- **NFR-03: Extensibility:** The system design should accommodate four major programs within the School of Computer Science and Engineering (Computer Science, Data Science, Computer Engineering, and Network Engineering) with the flexibility to add more programs in the future.

3.2.3 Use Case Scenarios

To validate the practical utility of the system, the following operational scenarios are defined:

3.2.4 Usage Scenarios

To validate the system’s practical usefulness, the following operational scenarios are identified:

- **Scenario 1: Scheduling supplementary classes for cases deviating from the standard curriculum (retaking, delayed study, accelerated study):** The academic planner needs to determine whether it is necessary to open a separate course section for a specific subject based on the demand for that subject, which is broken down to track in detail each demand from each cohort taking that subject, ensuring that no one is overlooked.
- **Scenario 2: Capacity planning for students (the “Standard” case):** It is necessary to determine the number of standard students who will take courses in the upcoming semester in order to open classes according to actual student demand and allocate human resources appropriately.
- **Scenario 3: Bottleneck identification:** The management observes a low graduation rate. The system identifies *Late Demand* to determine that a large number of students are getting stuck at which stage and in which courses, thereby addressing issues in bottleneck courses and promoting a review of the curriculum.

3.2.5 System Scope

The scope of this thesis is strictly defined by the following boundaries:

- **Data Domain:** The system focuses on academic data of the Faculty of Computer Science and Engineering, specifically including the majors within the Computer Science and Engineering (CSE) faculty: Computer Science (CS), Data Science (DS), Computer Engineering (CE), and Network Engineering (NE).
- **Technical Implementation:**
 - Designing a relational database schema (MySQL) that reflects the university’s academic regulations.
 - Developing a synthetic data generation tool based on Python.
 - Implementing complex SQL Stored Procedures for demand classification.

- Visualizing results through an interactive Business Intelligence dashboard.
- **Exclusions:** The prediction system estimates the *volume* of demand but does not handle *timetabling* (arranging specific time slots) or instructor assignment.

3.3 Algorithms and Mathematical Formalization

The core intelligence of the Course Demand Estimation System relies on a deterministic classification algorithm. Unlike black-box machine learning models, this system employs a transparent, rule-based inference engine. This section formalizes the logic used to categorize student demand into actionable groups and describes the constraint satisfaction algorithm used to simulate realistic registration behaviors.

3.3.1 Mathematical Formalization

To precisely define the estimation logic, we model the university’s academic environment using Set Theory.

Core Definitions

Let the academic domain be defined by the following sets and mappings:

- Let S be the set of all active students in the university.
- Let C be the set of all available courses in the curriculum.
- Let H_s be the academic history of a student $s \in S$, defined as a set of tuples (c, g, t) , where:
 - $c \in C$ is the course identifier.
 - $g \in [0, 100]$ is the final grade achieved.
 - $t \in \mathbb{N}$ represents the semester index.
- Let $\text{Pre}(c)$ denote the set of direct prerequisite courses for course c . If $\text{Pre}(c) = \emptyset$, the course has no prerequisites.
- Let $\text{Credits}(c)$ denote the credit value assigned to course c .
- **Course Enrollment Constraints:**
 - **Prerequisite:** If course A is a prerequisite for course B , student s must complete and pass A ($g \geq 50$) in a semester $t_A < t_B$.
 - **Co-requisite:** If A and B are co-requisites, they must be taken concurrently in the same semester ($t_A = t_B$).
 - **Previous:** If A is a "previous" requirement for B , the student must take A in a semester $t_A < t_B$, regardless of whether they pass or fail.

Helper Predicates

To facilitate the classification logic, we define the following formal boolean predicates:

1. Passing Condition (HasPassed) A student s is considered to have passed course c if their academic record contains an entry for that course with a grade of 50 or higher.

Algorithm 7 Condition for passing a course

$$\text{HasPassed}(s, c) \iff \exists(c, g, t) \in H_s : g \geq 50 \quad (3.1)$$

2. Enrollment Eligibility (IsEligible) A student is eligible to enroll in course c if they satisfy the mandatory prerequisite conditions of that course defined in $\text{Pre}(c)$. This logic is fundamental for curriculum dependency analysis.

Algorithm 8 Eligibility for course enrollment

$$\text{IsEligible}(s, c) \iff \forall p \in \text{Pre}(c), \text{HasPassed}(s, p) \quad (3.2)$$

3.3.2 Demand Classification Logic

The Total Demand for a specific course c in the target semester T_{target} , denoted as $D(c)$, is defined as the union of four disjoint subsets based on the students' academic status.

Retake Demand (D_{retake})

The retake segment identifies students who have previously attempted the course but failed, and have not yet successfully cleared it in subsequent attempts. In many academic frameworks, this group receives the highest priority during resource allocation to ensure timely graduation.

Algorithm 9 Retake Demand Identification

$$D_{\text{retake}}(c) = \{s \in S \mid \exists(c, g, t) \in H_s : (g < 50) \wedge \neg \text{HasPassed}(s, c)\} \quad (3.3)$$

Standard Demand (D_{standard})

This segment represents the regular progression of students according to the established curriculum roadmap. Let $\text{Path}(s, T_{\text{target}})$ be the set of courses prescribed for student s in the target semester based on their major's recommended schedule. This demand consists of students who are scheduled to take the course, are academically eligible to do so, and have not yet successfully completed it.

Algorithm 10 Standard Demand Identification

$$D_{\text{standard}}(c) = \{s \in S \mid c \in \text{Path}(s, T_{\text{target}}) \wedge \text{IsEligible}(s, c) \wedge \neg \text{HasPassed}(s, c)\} \quad (3.4)$$

Late Demand (D_{late})

Often referred to as “Backlog Demand,” this category includes students who were scheduled to take a course in a previous semester ($k < T_{\text{target}}$) but have not yet attempted it. This metric effectively identifies the “Technical Debt” in a student’s academic progression.

Algorithm 11 Late Demand (Backlog) Identification

$$D_{\text{late}}(c) = \{s \in S \mid \exists k < T_{\text{target}} : c \in \text{Path}(s, k) \wedge (\nexists (c, g, t) \in H_s) \wedge \text{IsEligible}(s, c)\} \quad (3.5)$$

Note: The condition $\nexists (c, g, t) \in H_s$ ensures that D_{late} is disjoint from D_{retake} . It captures only those students who omitted the course entirely, whereas D_{retake} captures those who attempted it but failed.

Fast Demand (D_{fast})

This segment identifies proactive or high-performing students who intend to accelerate their academic timeline by enrolling in courses scheduled for future semesters ($k > T_{\text{target}}$). This subset is essential for modeling “acceleration capacity” within the student body.

Algorithm 12 Fast Demand (Acceleration) Identification

$$D_{\text{fast}}(c) = \{s \in S \mid \exists k > T_{\text{target}} : c \in \text{Path}(s, k) \wedge \text{IsEligible}(s, c)\} \quad (3.6)$$

3.3.3 Constraint-Based Simulation Algorithm

Directly summing the cardinalities of the demand sets often leads to an overestimation, as it assumes students have an infinite capacity for credit hours. To ensure the model reflects reality, the system implements a **Greedy Resource Allocation Algorithm** to account for the maximum credit limit constraint.

Greedy Selection Process

For each student $s \in S$, we construct a potential course basket B_s , containing all courses for which the student is eligible, prioritized by their academic urgency:

Algorithm 13 Priority-Based Course Sorting

$$B_s = \text{SortPriority}(D_{\text{retake}} \cup D_{\text{late}} \cup D_{\text{standard}} \cup D_{\text{fast}}) \quad (3.7)$$

Priority Hierarchy: Retake $\succ$ Late $\succ$ Standard $\succ$ Fast. *Note: Late demand (backlog) is prioritized over standard demand to minimize “Technical Debt” and prevent further delays in graduation.*

The simulation selects a subset $B'_s \subseteq B_s$ by iterating through the sorted list and appending courses until the university’s credit limit L_{max} (e.g., 24 credits) is reached:

Algorithm 14 Student Course Basket Simulation

```
1:  $B'_s \leftarrow \emptyset$ 
2: CurrentCredits  $\leftarrow 0$ 
3: for  $c \in B_s$  do
4:   if CurrentCredits + Credits( $c$ )  $\leq L_{\max}$  then
5:      $B'_s \leftarrow B'_s \cup \{c\}$ 
6:     CurrentCredits  $\leftarrow$  CurrentCredits + Credits( $c$ )
7:   end if
8: end for
```

Final Demand Estimation

The final forecasted demand for a course c , denoted as $\text{FinalDemand}(c)$, is the aggregate count of all students whose constrained basket B'_s contains course c :

Algorithm 15 Final Aggregated Demand Calculation

$$\text{FinalDemand}(c) = \sum_{s \in S} \mathbb{I}(c \in B'_s) \quad (3.8)$$

where $\mathbb{I}(\cdot)$ is the indicator function, which equals 1 if the condition $c \in B'_s$ is true and 0 otherwise. This simulation ensures that the forecast reflects the actual number of seats likely to be consumed, accounting for credit limits rather than just student interest. This serves as a reliable decision-support metric for academic capacity planning and resource allocation.

Chapter 4

IMPLEMENTATION

4.1 Synthetic Data Generation

Due to strict privacy regulations (the university’s internal policies) regarding access to real student records, direct analysis of production data is not feasible. To address this issue, a powerful **Synthetic Data Generation Tool** was developed in Python. This tool ensures that the dataset is not merely random noise but instead creates a high-fidelity “Digital Twin” of the academic environment, reflecting statistical characteristics, temporal consistency, and the logical constraints of real-world operations.

4.1.1 Algorithmic Implementation Logic

The simulation is implemented via a nested loop structure: *For each Student* $\rightarrow$ *For each Semester*. The core logic within each iteration consists of four sequential phases.

Phase 1: Priority-Based Registration Queue

For every simulated semester, the algorithm constructs a prioritized “wish-list” of courses. This is implemented using a **Priority Queue** to reflect rational student behavior:

- **Priority 1 (Critical):** English bottleneck requirements and **Retake** courses (from the `failed_courses` set). This simulates the urgency to clear academic debt.
- **Priority 2 (Standard):** An “Accumulated Sweep” of the curriculum roadmap up to the current semester index, adding any course not yet passed.
- **Priority 3 (Acceleration):** For high-capacity students ($C_i > 0.75$) with a ‘FAST’ tag, the system adds advanced courses from future semesters.

Phase 2: Greedy Resource Allocation

Real-world registration is bounded by credit limits. The algorithm applies a **Greedy Approach** to the merged queue:

Algorithm 16 Course Selection Queue Assembly

$$Q_{\text{total}} = [P_1, P_2, P_3] \tag{4.1}$$

Where P_1, P_2, P_3 represent the prioritized subsets for Retakes/English, Standard Roadmap, and Acceleration respectively.

The system iterates through Q_{total} and accepts courses until the cumulative credits reached `MAX_CREDITS` (24). Any subsequent courses in the queue are deferred to the following semester.

Phase 3: Stochastic Grade Generation

The final grade $G_{i,j}$ for student i in course j is calculated using a deterministic performance base adjusted by stochastic variables:

Algorithm 17 Stochastic Grade Generation Logic

$$G_{i,j} = \text{clip}((C_i \times D_j \times 100) + \epsilon + \beta, 0, 100) \quad (4.2)$$

Where:

- $\text{clip}(x, a, b)$: A function constraining x to the interval $[a, b]$.
- $\epsilon \sim \mathcal{U}(-10, 10)$: Random noise representing situational variance.
- β : **Retake Boost**. $\beta = 15$ if $c \in \text{failed_courses}$, otherwise $\beta = 0$, simulating the advantage of prior exposure to the material.
- **Accident Factor**: A Bernoulli trial with $p = 0.03$ that overrides the result to a failing grade ($G < 45$), modeling unforeseen personal or health-related disruptions.

Phase 4: State Update

Following grade assignment, the student's academic record H_s and state sets are updated:

- **Success** ($G \geq 50$): Course is added to `passed_courses` and removed from `failed_courses`.
- **Failure** ($G < 50$): Course is added to `failed_courses`, triggering its inclusion in Priority 1 for the subsequent semester loop.

4.1.2 Code Implementation Snippet

The following listing demonstrates the Python implementation of the priority queuing and grading logic described above.

Module 1: System Configuration

The simulation environment is initialized with global constants that define the temporal bounds and student population characteristics.

- **Temporal Bounds**: Set to Semester 1, 2025–2026.
- **Institutional Mapping**: `PROGRAM_MAP` and `PROGRAM_WEIGHTS` define the distribution of students across majors (DS, CE, NE, CS).
- **Difficulty Scaling**: `DIFF_CONFIG` maps qualitative labels (e.g., 'VERY_HARD') to quantitative coefficients D_j .

```

CURRENT_YEAR_INT = 2025
CURRENT_SEM_INT = 2

DIFF_CONFIG = {
    'VERY_HARD': (0.70, 0.90),
    'HARD':      (0.90, 1.10),
    'MEDIUM':   (1.10, 1.25),
    'EASY':      (1.25, 1.40),
    'VERY_EASY': (1.40, 1.50)
}

```

Listing 4.1: Configuration and Difficulty Constants

Module 2: Student Attribute Initialization

For each cohort, students are generated with unique profiles. The code differentiates between **Active Cohorts** and **Delayed Cohorts** using the probability distributions defined in the methodology.

```

# Normal distribution for active cohorts
if batch <= 22: # Delayed cohorts
    base_cap = random.choices([random.uniform(0.65, 0.75),
                              random.uniform(0.75, 0.9)], weights
                              =[50, 50], k=1)[0]
else: # Active cohorts
    base_cap = random.gauss(0.80, 0.15)

capacity = max(0.2, min(0.99, base_cap + cap_mod))

```

Listing 4.2: Capacity and Pace Assignment

Module 3: Prioritized Enrollment and Credit Control

The simulation implements the three-tier priority system. The "Accumulated Sweep" logic (Priority 2) ensures students attempt all courses from their roadmap up to the current semester index.

```

# Phase 1: Priority Construction
priority_1 = english_queue + list(failed_courses)
priority_2 = [c for c in roadmap if c not in passed_courses]

# Phase 2: Greedy Credit Check
for p_list in [priority_1, priority_2, priority_3]:
    for c in p_list:
        cred = COURSE_CREDITS.get(c, 3)
        if current_credit_load + cred <= MAX_CREDITS:
            final_registration.append(c)
            current_credit_load += cred

```

Listing 4.3: Priority Queue and Greedy Allocation

Module 4: Grading Logic and State Updates

The grade generation uses the formula $G = (Capacity \times Difficulty \times 100) + \text{Noise}$. It also includes the "Accident Factor" to simulate outlier failures.

```

# Phase 3: Grading
diff = get_difficulty_coefficient(course_id)
base = (capacity + (0.15 if retake else 0)) * 100
final_raw = int(base * diff + random.randint(-10, 10))

# Accident Factor
if random.random() < 0.03: final_raw = random.randint(0, 45)

# Phase 4: State Update
if total >= 50:
    passed_courses.add(course_id)
    if course_id in failed_courses: failed_courses.remove(course_id)
else:
    failed_courses.add(course_id)

```

Listing 4.4: Grade Generation and History Update

To ensure database performance, results are exported in "chunks" of 500 records per INSERT statement, maintaining foreign key integrity during the process.

4.2 Database Schema Adjustments

Since the core database was provided as a pre-configured SQL script, this section details the specific refinements and table additions required to support the simulation engine and ensure referential integrity for the synthetic data.

4.2.1 New Table Implementation: `enrollment_batch`

A new table, `enrollment_batch`, was introduced to serve as a bridge between students, their majors (`program`), and their specific academic roadmaps (`pathway`). This structure allows the simulation to assign different curriculum paths to students based on their English proficiency and entry year.

```

DROP TABLE IF EXISTS 'enrollment_batch';
CREATE TABLE 'enrollment_batch' (
    'id' VARCHAR(50),
    'program_id' INT NOT NULL,
    'pathway_id' INT NOT NULL,
    PRIMARY KEY ('id'),
    FOREIGN KEY ('program_id') REFERENCES 'program' ('id'),
    FOREIGN KEY ('pathway_id') REFERENCES 'pathway' ('id')
);

```

Listing 4.5: Implementation of the `enrollment_batch` table

4.2.2 Modified Table: `student`

The original `student` table was adjusted to incorporate the `enrollment_batch_id` foreign key. This modification ensures that each student is strictly bound to a valid enrollment batch, enabling cascading updates and deletions to maintain data consistency.

```

DROP TABLE IF EXISTS 'student';
CREATE TABLE 'student' (

```

```

'id' varchar(50) NOT NULL,
'name' varchar(255) DEFAULT NULL,
'major' varchar(45) DEFAULT NULL,
'batch' int DEFAULT NULL,
'enrollment_batch_id' VARCHAR(20),
PRIMARY KEY ('id'),
-- Adjustment: Establishing link to the enrollment_batch system
-- with ON DELETE CASCADE and ON UPDATE CASCADE for integrity
FOREIGN KEY ('enrollment_batch_id')
REFERENCES 'enrollment_batch'('id')
ON DELETE CASCADE
ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

```

Listing 4.6: Adjusted student table with referential integrity constraints

4.2.3 Data Integrity Enforcement

To ensure the reliability of the estimation, strict database constraints are enforced at the schema level:

Referential Integrity (Foreign Keys)

The system utilizes InnoDB’s foreign key constraints to prevent ”orphan records.”

- A `result` cannot be created for a non-existent `student_id`.
- A `class_session` cannot refer to a non-existent `course_id`.

This ensures that the synthetic data generation script (Chapter 4.1) must generate entities in a strict topological order: *Courses* → *Classes* → *Students* → *Results*.

Logic Constraints and Data Validation

Beyond standard SQL constraints, the data population process enforces temporal logic:

- **Temporal Consistency:** A student from batch K24 (Admission 2024) is strictly prevented from having grade records dated in 2023.
- **Duplicate Prevention:** The composite primary key on the `result` table (`student_id`, `class_id`) prevents the accidental insertion of duplicate grades for the same course session, ensuring that GPA calculations and pass/fail counts remain accurate.

4.3 Logic Implementation: Demand Estimation Engine

The core value of the Course Demand Estimation System is derived from its capability to transform raw academic records into actionable insights. Unlike ”black-box” machine learning models, this transformation is executed by a deterministic and transparent logic engine, implemented using optimized SQL Stored Procedures.

The estimation logic is encapsulated within the procedure `Refresh_Demand_Dashboard`. This procedure orchestrates a multi-stage calculation process, systematically classifying student demand into four distinct categories: *Retake*, *Standard*, *Late*, and *Fast*.

4.3.1 Output Structure: The Denormalized Dashboard Table

To optimize query latency for the frontend dashboard, the system avoids executing complex joins on the relational schema in real-time. Instead, it utilizes a **Materialization Strategy**, pre-calculating demand metrics and storing the results in a denormalized table named `dashboard_demand_data`.

- **Granularity:** Each record represents the aggregated demand for a specific unique course (c) within the target semester (T_{target}).
- **Key Attributes:**
 - `total_students` (Volume): The total count of projected enrollments.
 - `batches_involved` (Cohort Distribution): A serialized string listing the student cohorts driving this demand (e.g., "K21, K22"), enabling administrators to identify specific pressure points.

4.3.2 Phase 1: Retake Demand Logic (High Priority)

The first module specifically targets students who have previously attempted a course but failed, requiring immediate remediation. Prioritizing this segment is essential for "clearing debts" and preventing bottlenecks that lead to graduation delays.

Algorithmic Logic

The identification mechanism applies a **Set-Theoretic Difference** operation. We define the Retake Demand for a course c as the set of students who failed c , excluding those who successfully cleared it in a subsequent semester:

Algorithm 18 Retake Demand Identification Logic

$$D_{\text{retake}}(c) = \{s \in \text{Failed}(c)\} \setminus \{s \in \text{Passed_Later}(c)\} \quad (4.3)$$

Mathematically, the algorithm filters for result records where the average grade is below 50 (< 5.0) and enforces a constraint that the student ID must not appear in the set of successful completions for that same course.

The following SQL procedure implements this logic using a 'NOT IN' subquery to exclude false positives (students who failed initially but passed later).

1. The Subquery: Identifying the "Success Set" The following inner query creates a "White-list" of all students who have achieved a passing grade (≥ 50) at any point in their history for the specific course:

```
SELECT r2.student_id
FROM result r2
JOIN class_session cs2 ON r2.class_id = cs2.id
WHERE cs2.course_id = cs.course_id AND r2.avg >= 50
```

- **Objective:** To identify students who have already satisfied the course requirements, making them ineligible for the Retake Demand category.

2. The Main Query: Applying the Exclusion Filter The primary query identifies historical failures and then cross-references them with the "Success Set":

- `WHERE r.avg < 50`: Initially targets all student records where a course was failed.
- `AND s.id NOT IN (...)`: This is the critical logic gate. The system checks: *"If a student failed previously but subsequently passed, their ID will exist in the subquery and be excluded from the demand count."* Only those without any record of a pass are retained.

3. Aggregation and Validation Finally, the system aggregates the filtered results to provide a clean metric for the dashboard:

- `COUNT(DISTINCT s.id)`: Ensures mathematical accuracy. If a student failed the same course multiple times, they are counted as a single "unit of demand" (one required seat).
- `HAVING ... > 0`: Optimizes the dashboard by suppressing courses with zero current demand.

SQL Component	Functional Role in Estimation
<code>WHERE r.avg < 50</code>	Extracts the raw historical backlog of all failed attempts.
Subquery (<code>>= 50</code>)	Defines the set of students who have successfully cleared the course.
<code>NOT IN</code>	Executes the set-theoretic difference: $Students_{Failed} \setminus Students_{Passed}$.
<code>DISTINCT s.id</code>	Prevents overestimation by ensuring each student represents only one seat.

Table 4.1: Logic decomposition of the Retake Demand SQL query.

The complete SQL implementation, integrating the join logic and filtering mechanisms discussed above, is provided in the following script:

```
-- Logic: Identify failures (avg < 50) AND exclude subsequent passes
INSERT INTO dashboard_demand_data (demand_type, course_id,
course_name, batch, student_count, target_year, target_sem)
SELECT
    'RETAKE', c.id, c.name, s.batch, COUNT(DISTINCT s.id), p_Year,
    p_Sem
FROM result r
JOIN student s ON r.student_id = s.id
JOIN class_session cs ON r.class_id = cs.id
JOIN course c ON cs.course_id = c.id
WHERE r.avg < 50
-- Critical: Set Difference Check
AND s.id NOT IN (
    SELECT r2.student_id
    FROM result r2
    JOIN class_session cs2 ON r2.class_id = cs2.id
```

```

WHERE cs2.course_id = cs.course_id
      AND r2.avg >= 50
)
GROUP BY c.id, c.name
HAVING COUNT(DISTINCT s.id) > 0;

```

Listing 4.7: SQL Implementation for Retake Demand

4.3.3 Phase 2: Standard Demand Logic (Curriculum Alignment)

Standard Demand module identifies students who are progressing according to their major’s prescribed roadmap. The primary goal is to forecast enrollment for courses that students are ”expected” to take in the upcoming semester.

Algorithmic Logic

The system maps the student’s admission batch (e.g., K23) to the curriculum timeline using a temporal formula. The identification of a specific course c within a student’s curriculum pathway at the target time T_{target} is governed by two primary constraints:

- **Formula:** The target curriculum year/semester is derived from the difference between the Forecast Year and the Student’s Batch.
- **Filtering:** The logic excludes students who have already passed the course (e.g., via previous summer terms or fast-track options) using a `NOT EXISTS` clause.

Algorithm 19 Standard Demand Identification Logic

$$D_{\text{standard}}(c) = s \in S \mid c \in \text{Path}(s, T_{\text{target}}) \wedge \neg \text{HasPassed}(s, c) \quad (4.4)$$

1. Dynamic Academic Year Mapping The most critical part of this module is the mathematical conversion of the target date into a relative curriculum index:

```

WHERE cp.'year' = FLOOR( ( ((p_Year - (2000 + s.batch)) * 2 + p_Sem) +
1 ) / 2 )
AND cp.'semester' = p_Sem

```

- **Logic:** This formula calculates the cumulative number of semesters elapsed since the student’s entry ($2000 + s.\text{batch}$). By normalizing this value, the system can pinpoint exactly which year of the roadmap the student should currently be attending.

2. Eligibility and Necessity Constraints To ensure data quality, the query applies the following filters:

- **Seniority Bound:** The condition `BETWEEN 0 AND 6` limits the calculation to active students within a standard 7-year graduation window.
- **Completion Check:** Using a `NOT EXISTS` subquery, the system verifies that the student does not have a passing grade (≥ 50) for the course, ensuring the demand represents a new enrollment.

SQL Component	Functional Role in Estimation
JOIN course_pathway	Joins the student to their specific major and English-level roadmap.
FLOOR(...) calculation	Converts real-world calendar dates into relative academic years (Year 1–4).
NOT EXISTS	Excludes students who have already earned credit for the course.
GROUP BY s.batch	Segregates demand by cohort (e.g., K22, K23) for granular capacity planning.

Table 4.2: Logic decomposition of the Standard Demand SQL query.

The complete SQL script for the Standard Demand module is as follows:

```
-- Logic: Identify students on-track based on their curriculum roadmap
INSERT INTO dashboard_demand_data (
    demand_type, course_id, course_name, batch,
    student_count, target_year, target_sem
)
SELECT
    'STANDARD',
    c.id,
    c.name,
    s.batch,
    COUNT(DISTINCT s.id),
    p_Year,
    p_Sem
FROM student s
JOIN enrollment_batch eb ON s.enrollment_batch_id = eb.id
JOIN course_pathway cp ON eb.program_id = cp.program_id
    AND eb.pathway_id = cp.pathway_id
JOIN course c ON cp.course_id = c.id
WHERE cp.'year' = FLOOR( ((p_Year - (2000 + s.batch)) * 2 + p_Sem) +
    1 ) / 2 )
    AND cp.'semester' = p_Sem
-- Filter for students within the standard 7-year window
AND (p_Year - (2000 + s.batch)) BETWEEN 0 AND 6
-- Exclude students who have already passed the course
AND NOT EXISTS (
    SELECT 1 FROM result r
    JOIN class_session cs ON r.class_id = cs.id
    WHERE r.student_id = s.id
        AND cs.course_id = c.id
        AND r.avg >= 50
)
GROUP BY c.id, c.name, s.batch;
```

Listing 4.8: SQL Implementation for Standard Demand

4.3.4 Phase 3: Late Demand Logic

This module manages the "Backlog" demand, identifying courses from previous semesters that a student has not yet cleared. Due to the complexity of academic regulations, this

phase employs a **Constraint-Based Simulation** approach rather than simple filtering.

Algorithmic Strategy

The logic treats the course selection as a resource-constrained optimization problem (similar to the Knapsack Problem, solved here via a Greedy strategy). The process involves two critical checks:

- **Dependency Graph Traversal (Prerequisites):** Before a backlog course is added to the candidate list, the system must verify that all its predecessors (Prerequisites) have been passed.
- **Cumulative Resource Allocation (Credit Limit):** Courses are prioritized by their order in the curriculum. The system simulates adding courses to the student's basket until the cumulative credit count reaches the maximum limit ($C_{\max}$).

The solution leverages SQL Common Table Expressions (CTEs) to separate the candidate identification from the resource simulation. The implementation utilizes Common Table Expressions (CTEs) to separate the candidate identification from the resource simulation.

1. Prerequisite Graph Traversal (The Recursive Check) The most critical constraint in academic planning is the prerequisite chain. The logic must ensure a student cannot register for a course unless all predecessors are satisfied.

First, we identify if a student has successfully passed a specific prerequisite course (C_{pre}):

```
-- Inner Logic: Check if Student S passed Course C_pre
SELECT 1 FROM result r_pre
JOIN class_session cs_pre ON r_pre.class_id = cs_pre.id
WHERE r_pre.student_id = s.id
      AND cs_pre.course_id = rel.course_id2 -- The Prerequisite ID
      AND r_pre.avg >= 50
```

Next, we wrap this in a **Double Negation** logic structure. We search for any "blocking" relationship—a prerequisite that exists in the curriculum but does **not** exist in the student's passed history. If no such blocking relationship is found, the student is eligible.

```
-- Outer Logic: Ensure NO unmet prerequisite exists
AND NOT EXISTS (
  SELECT 1 FROM course_course_relationship rel
  WHERE rel.course_id1 = c.id          -- The Target Course
        AND rel.relationship_id = 1    -- Type: Prerequisite
        -- The "Blocking" Condition: Prereq exists but Result does NOT
        AND NOT EXISTS (
          SELECT 1 FROM result r_pre
          JOIN class_session cs_pre ON r_pre.class_id = cs_pre.id
          WHERE r_pre.student_id = s.id
                AND cs_pre.course_id = rel.course_id2
                AND r_pre.avg >= 50
        )
  )
)
```

2. Temporal Validity Check The system must filter the search space to include only courses that should have been completed in the past relative to the student’s current timeline.

```
WHERE (p_Year - (2000 + s.batch)) BETWEEN 0 AND 9
-- Compare Course Index vs. Student Current Index
AND ((cp.'year' - 1) * 2 + cp.'semester') <= ((p_Year - (2000 + s.
    batch)) * 2 + p_Sem)
```

- **Index Calculation:** The formula $((Year - 1) \times 2 + Sem)$ converts the academic timeline into a linear integer index (e.g., Year 1 Sem 1 = 1, Year 2 Sem 1 = 3).
- **Comparison:** We only consider courses where $Index_{Course} \leq Index_{Student}$, ensuring we don’t accidentally suggest future courses as backlog.

3. Resource Simulation (Greedy Algorithm) Once the valid candidate courses are identified, we apply a **Greedy Strategy** to fill the student’s ”shopping basket” up to the credit limit. This is achieved using an SQL Window Function:

```
Sim_Late AS (
    SELECT *,
        -- Calculate Running Total of credits per student
        SUM(C_Credit) OVER (
            PARTITION BY Student_ID
            ORDER BY C_Idx ASC, C_ID ASC
        ) AS Acc_Credits
    FROM Student_Backlog
)
```

- **PARTITION BY Student_ID:** The calculation resets for each unique student.
- **ORDER BY C_Idx ASC:** The simulation prioritizes ”older” debts. A Year 1 backlog course is added to the basket before a Year 2 backlog course.
- **Acc_Credits:** This column represents the cumulative load at each step.

4. Capacity Filtering Finally, the system selects the courses that fit within the constraints.

```
'LATE', C_ID, C_Name, batch_num, COUNT(DISTINCT Student_ID), p_Year,
    p_Sem
FROM Sim_Late
WHERE Acc_Credits <= p_MaxCredits
    AND C_Idx < S_Curr_Idx
```

- **Acc_Credits <= p_MaxCredits:** This acts as the ”cut-off” point. If a student needs 5 backlog courses but only has credit capacity for 3, the window function ensures only the first 3 (most urgent) are selected.
- **C_Idx < S_Curr_Idx:** This strictly defines the demand as ”Late.” If the course index equals the current index, it falls under Standard Demand, not Backlog.

SQL Component	Functional Role in Simulation
Student_Backlog	Defines the "Search Space" of all unpassed past courses.
relationship_id = 1	Enforces the Directed Acyclic Graph (DAG) logic for prerequisites.
SUM(...) OVER	Implements the Greedy Algorithm to accumulate credits row-by-row.
Acc_Credits <= Limit	The stopping condition for the resource simulation.

Table 4.3: Logic decomposition of the Late Demand Simulation.

Identifies courses from previous curriculum semesters ($C_Idx < S_Curr_Idx$) that were missed. The complete logic, integrating the prerequisite graph traversal and the greedy resource simulation, is presented below:

```

WITH Student_Backlog AS (
    SELECT s.id AS Student_ID, s.batch AS batch_num, c.id AS C_ID,
           c.name AS C_Name,
           (c.credit_theory + c.credit_lab) AS C_Credit,
           (cp.'year' - 1) * 2 + cp.'semester' AS C_Idx,
           (p_Year - (2000 + s.batch)) * 2 + p_Sem AS S_Curr_Idx
    FROM student s
    JOIN enrollment_batch eb ON s.enrollment_batch_id = eb.id
    JOIN course_pathway cp ON eb.program_id = cp.program_id AND eb
    .pathway_id = cp.pathway_id
    JOIN course c ON cp.course_id = c.id
    WHERE (p_Year - (2000 + s.batch)) BETWEEN 0 AND 9
           AND ((cp.'year' - 1) * 2 + cp.'semester') <= ((p_Year -
(2000 + s.batch)) * 2 + p_Sem)
           AND NOT EXISTS (SELECT 1 FROM result r JOIN class_session
cs ON r.class_id = cs.id WHERE r.student_id = s.id AND cs.
course_id = c.id)
           AND NOT EXISTS (SELECT 1 FROM course_course_relationship
rel WHERE rel.course_id1 = c.id AND rel.relationship_id = 1 AND
NOT EXISTS (SELECT 1 FROM result r_pre JOIN class_session cs_pre
ON r_pre.class_id = cs_pre.id WHERE r_pre.student_id = s.id AND
cs_pre.course_id = rel.course_id2 AND r_pre.avg >= 50))
),
Sim_Late AS (
    SELECT *, SUM(C_Credit) OVER (PARTITION BY Student_ID ORDER BY
C_Idx ASC, C_ID ASC) AS Acc_Credits FROM Student_Backlog
)
SELECT
    'LATE', C_ID, C_Name, batch_num, COUNT(DISTINCT Student_ID),
    p_Year, p_Sem
FROM Sim_Late WHERE Acc_Credits <= p_MaxCredits AND C_Idx <
S_Curr_Idx
GROUP BY C_ID, C_Name, batch_num;

```

Listing 4.9: SQL Simulation for Late Demand (Prerequisite & Credit Check)

4.3.5 Phase 4: Fast Demand Logic (Acceleration Strategy)

The final component of the estimation engine addresses "Fast Demand"—cases where high-performing students attempt to register for future courses ahead of their scheduled roadmap. This behavior is typical of students aiming to graduate early or clear prerequisites for advanced electives.

Algorithmic Strategy

To model this accurately, we cannot simply list all future courses. We must apply a **Resource-Constrained Forward Look-Ahead** strategy. The logic operates on a "Rational Agent" assumption:

"A student will first prioritize their current standard semester courses. Only if they have remaining credit capacity ($C_{remaining} > 0$) will they attempt to register for future courses."

Logic Decomposition

The implementation mirrors the simulation structure of Late Demand but with inverted temporal constraints.

1. The Forward Search Window Unlike Backlog (which looks at history), this phase scans a specific window of future opportunity:

```
WHERE ((cp.'year' - 1) * 2 + cp.'semester') >= ((p_Year - (2000 + s.
    batch)) * 2 + p_Sem)
AND ((cp.'year' - 1) * 2 + cp.'semester') <= ((p_Year - (2000 + s.
    batch)) * 2 + p_Sem) + 2
```

- **Lower Bound ($\geq$ Current):** Includes the current semester to calculate the base load.
- **Upper Bound ($\leq$ Current + 2):** Limits the "acceleration" horizon to 1 academic year (2 semesters). This prevents unrealistic jumps (e.g., a Freshman trying to take a Senior project).

2. Prerequisite Enforcement Acceleration is strictly bound by academic dependencies. The NOT EXISTS block ensures that a student cannot "skip" a prerequisite just because they want to learn faster.

- **Mechanism:** The logic verifies that for any future course C_{future} , all its prerequisites C_{pre} are already in the Passed set.

3. Simulation and Capacity Filling The simulation integrates both Standard and Future courses into a single priority queue to determine which acceleration courses "make the cut."

```
Sim_Fast AS (
    SELECT *,
        CASE WHEN C_Idx = S_Target_Idx THEN 'ST' ELSE 'FAST' END AS
    D_Grp,
        SUM(C_Credit) OVER (
```

```

        PARTITION BY Student_ID
        ORDER BY C_Idx ASC, C_ID ASC
    ) AS Acc_Credits
FROM Student_Backlog_Fast
)

```

- **Prioritization (ORDER BY C_Idx ASC):** The system naturally orders current courses before future courses.
- **Credit Saturation:** The SUM() function fills the student's credit bucket with Standard courses first. If the bucket (*p_MaxCredits*) is not full, the remaining space is allocated to the "FAST" courses.

4. Final Extraction The final query filters the results to report only the acceleration component:

```
WHERE Acc_Credits <= p_MaxCredits AND D_Grp = 'FAST'
```

This ensures we don't double-count Standard Demand (which was calculated in Phase 2) while guaranteeing that the Fast Demand is actually feasible within the credit limit.

SQL Component	Functional Role in Simulation
Target_Idx <= Current + 2	Defines a realistic "Look-Ahead Window" (max 2 semesters), preventing students from jumping too far ahead.
ORDER BY C_Idx ASC	Enforces the "Rational Agent" rule: Credits are allocated to Standard courses <i>before</i> Fast courses.
CASE WHEN ... 'FAST'	Classification logic that distinguishes between "On-Track" duties and "Acceleration" desires.
D_Grp = 'FAST'	A final isolation filter to ensure the reported demand is strictly for acceleration (avoiding double-counting).

Table 4.4: Logic decomposition of the Fast Demand Simulation.

The complete SQL logic for the Fast Demand simulation is presented below:

```

INSERT INTO dashboard_demand_data (
    demand_type, course_id, course_name, batch,
    student_count, target_year, target_sem
)
WITH Student_Backlog_Fast AS (
    -- Step 1: Identify Candidate Courses (Current + Future Window)
    SELECT
        s.id AS Student_ID,
        s.batch AS batch_num,
        c.id AS C_ID,
        c.name AS C_Name,
        (c.credit_theory + c.credit_lab) AS C_Credit,
        (cp.'year' - 1) * 2 + cp.'semester' AS C_Idx,
        (p_Year - (2000 + s.batch)) * 2 + p_Sem AS S_Target_Idx
    FROM student s
    JOIN enrollment_batch eb ON s.enrollment_batch_id = eb.id

```

```

JOIN course_pathway cp ON eb.program_id = cp.program_id
AND eb.pathway_id = cp.pathway_id
JOIN course c ON cp.course_id = c.id
WHERE (p_Year - (2000 + s.batch)) BETWEEN 0 AND 6
-- Temporal Window: From Current Sem to Current + 2 Semesters
AND ((cp.'year' - 1) * 2 + cp.'semester') >= ((p_Year - (2000
+ s.batch)) * 2 + p_Sem)
AND ((cp.'year' - 1) * 2 + cp.'semester') <= ((p_Year - (2000
+ s.batch)) * 2 + p_Sem) + 2
-- Existence Check: Not yet passed
AND NOT EXISTS (
SELECT 1 FROM result r JOIN class_session cs ON r.class_id
= cs.id
WHERE r.student_id = s.id AND cs.course_id = c.id
)
-- Prerequisite Check
AND NOT EXISTS (
SELECT 1 FROM course_course_relationship rel
WHERE rel.course_id1 = c.id AND rel.relationship_id = 1
AND NOT EXISTS (
SELECT 1 FROM result r_pre
JOIN class_session cs_pre ON r_pre.class_id = cs_pre
.id
WHERE r_pre.student_id = s.id
AND cs_pre.course_id = rel.course_id2
AND r_pre.avg >= 50
)
)
),
Sim_Fast AS (
-- Step 2: Greedy Allocation (Prioritize Standard -> Fast)
SELECT *,
CASE WHEN C_Idx = S_Target_Idx THEN 'ST' ELSE 'FAST' END AS
D_Grp,
SUM(C_Credit) OVER (
PARTITION BY Student_ID
ORDER BY C_Idx ASC, C_ID ASC
) AS Acc_Credits
FROM Student_Backlog_Fast
)
-- Step 3: Extract only Feasible Acceleration Demand
SELECT
'FAST', C_ID, C_Name, batch_num,
COUNT(DISTINCT Student_ID), p_Year, p_Sem
FROM Sim_Fast
WHERE Acc_Credits <= p_MaxCredits
AND D_Grp = 'FAST' -- Only capture the acceleration portion
GROUP BY C_ID, C_Name, batch_num;

```

Listing 4.10: Full SQL Implementation for Fast Demand (Acceleration)

4.4 Dashboard Implementation

The implementation of the user interface and the Analytics Layer is built upon the **Microsoft Power BI** platform. The system serves as a critical bridge, transforming the complex logic of the estimation engine into intuitive visualizations, thereby supporting the Faculty Board and the Academic Affairs Office in strategic course planning.

4.4.1 Data Materialization Strategy

To decouple the complex computation logic from the visualization layer, the system avoids connecting Power BI directly to the normalized transaction tables (Direct-Query). Instead, it employs a Data Materialization approach.

The results of the four demand estimation phases (Retake, Standard, Late, Fast) are aggregated and inserted into a dedicated denormalized table: `dashboard_demand_data`. This table acts as a high-performance Data Mart for the dashboard.

```
CREATE TABLE dashboard_demand_data (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    -- Core Dimension: The classification of demand (e.g., 'RETAKE', '  
    FAST')  
    demand_type VARCHAR(20),  
    -- Course Dimensions  
    course_id VARCHAR(255),  
    course_name VARCHAR(255),  
    -- Student Cohort Dimension  
    batch INT,  
    -- Measures (Facts)  
    student_count INT,  
    -- Temporal Target Dimensions  
    target_year INT,  
    target_sem INT,  
    -- Optimization Indices for fast retrieval  
    INDEX (demand_type),  
    INDEX (course_id),  
    INDEX (batch),  
    INDEX (target_year, target_sem)  
);
```

Listing 4.11: Schema Definition for the Dashboard Data Mart

- **Pre-Aggregation:** By calculating `student_count` at the database level, we reduce the computational load on the BI tool.
- **Indexing Strategy:** Composite indices on (`target_year`, `target_sem`) ensure that filtering dashboard views by semester is near-instantaneous.

4.4.2 Data Visualization Strategy

The dashboard is designed following the Visual Information Seeking Mantra: *"Overview first, zoom and filter, then details-on-demand."* The visualization strategy is categorized into the following key components:

Overall Demand Classification (Donut Chart)

To provide an immediate, strategic overview of the student body's academic progression status, the dashboard utilizes a Donut Chart visualization, as shown in Figure 4.1.

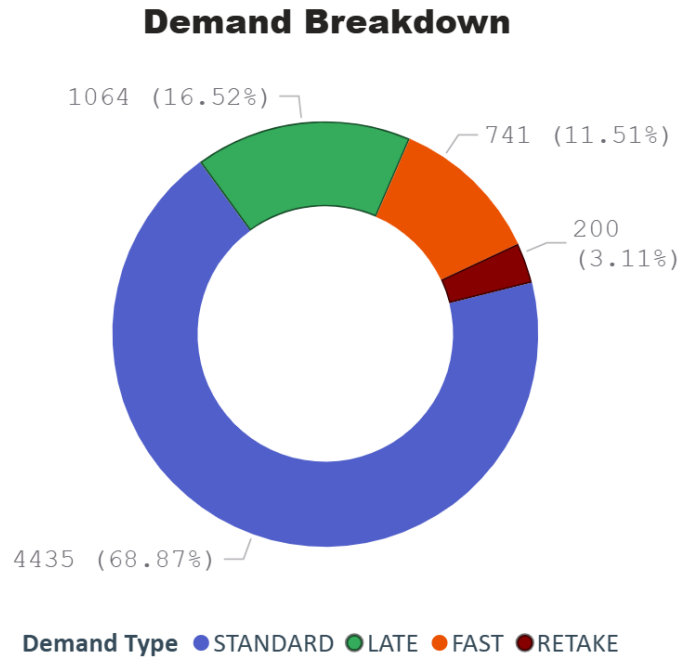


Figure 4.1: Distribution of Total Demand by Category (Standard, Late, Fast, Retake).

- **Visual Representation:** Unlike traditional views that break down demand by individual courses, this chart aggregates the total forecasted seats into four distinct logical categories defined by the estimation engine:
 - **Standard (Blue):** Represents students following the curriculum on-track (approx. 68.9%).
 - **Late (Green):** Represents backlog demand from previous semesters (approx. 16.5%).
 - **Fast (Orange):** Represents students accelerating their study plan (approx. 11.5%).
 - **Retake (Dark Red):** Represents immediate remediation needs due to failure (approx. 3.1%).
- **Analytical Value:** This visualization serves as a critical "Health Monitor" for the faculty's curriculum.
 - A dominant percentage of **Standard** demand indicates that the majority of students are successfully adhering to the prescribed roadmap.
 - Conversely, significant proportions of **Late** or **Retake** demand alert administrators to potential systemic bottlenecks, overly stringent prerequisite chains, or specific courses with historically high failure rates requiring academic intervention.
- **Interactivity (Cross-filtering):** The Donut Chart functions as the central high-level filter for the dashboard. Selecting a specific segment (e.g., clicking on the red *Retake* slice) instantly applies a context filter across the entire report. This updates subsequent tables and lists to display only the specific courses and students contributing to that category, facilitating targeted analysis.

Granular Analysis: Top 10 High-Demand Courses (Stacked Bar Chart)

While the Donut Chart provides a strategic overview, the "Top 10 Highest Course Demand" visualization (Figure 4.2) shifts the focus to tactical resource planning. This Stacked Horizontal Bar Chart ranks courses by total projected registration volume while simultaneously revealing the composition of that demand.

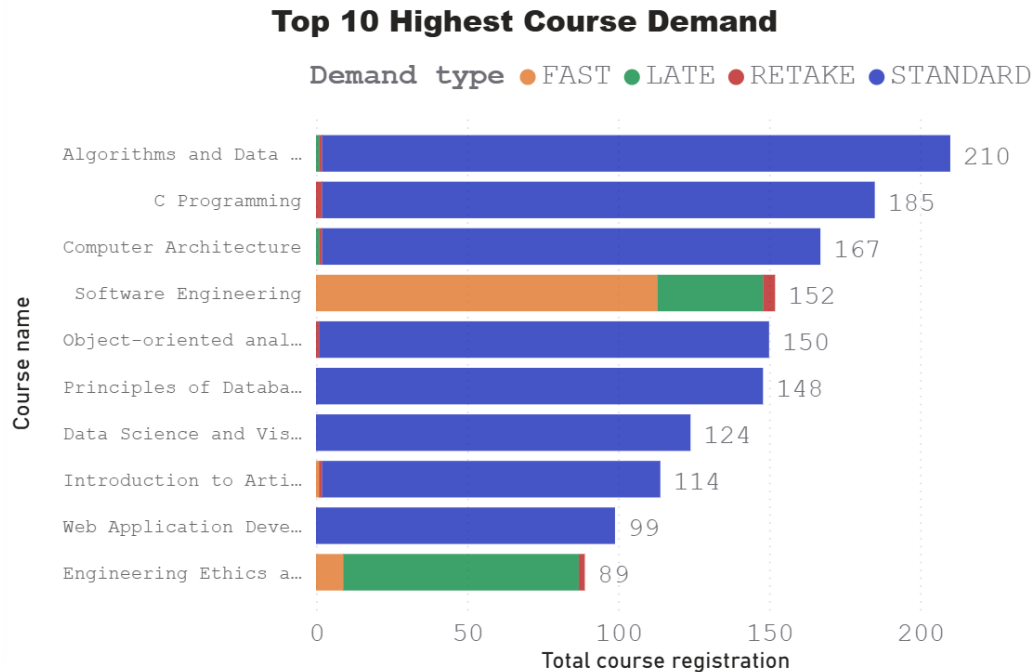


Figure 4.2: Top 10 Courses with Highest Projected Enrollment, segmented by Demand Type.

- **Visual Structure:** The chart sorts courses in descending order of total student count. Each bar is segmented by color to represent the four demand types (Standard, Late, Fast, Retake), consistent with the color coding established in the previous section.
- **Analytical Insights & Anomaly Detection:**
 - **Standard-Dominant Courses:** Courses such as *"Algorithms and Data Structures"* (210 students) and *"C Programming"* (185 students) show nearly solid blue bars. This indicates that the demand is driven almost entirely by students following the standard curriculum. For these courses, the primary administrative task is simply securing large lecture halls.
 - **Heterogeneous Demand Profiles:** A critical insight appears in the *"Software Engineering"* course (152 students). Unlike the others, its demand is composed largely of **Fast Demand (Orange)** and **Late Demand (Green)**, with very little Standard demand. This suggests a deviation from the typical roadmap—perhaps high-performing juniors are taking it early, or seniors are clearing it late. This requires a different scheduling strategy, possibly avoiding conflicts with both lower-year and upper-year courses.
 - **Backlog Indicators:** The course *"Engineering Ethics"* (89 students) shows a significant green segment (**Late Demand**). This signals that a large por-

tion of students are delaying this non-technical module until later semesters, creating a "bottleneck" effect that advisors may need to address.

- **Operational Value:** By visualizing the *composition* of demand, the Faculty Board can differentiate between "routine scheduling" (Standard demand) and "intervention scheduling" (Complex mixes of Fast/Late demand) to optimize class sectioning.

Cohort-to-Demand Flow Analysis (Sankey Diagram)

To understand how demand behavior evolves as students progress through their academic tenure, the dashboard employs a Sankey Diagram (Figure 4.3). This visualization maps the flow of students from their respective cohorts (Batches) on the left to the specific demand categories on the right.

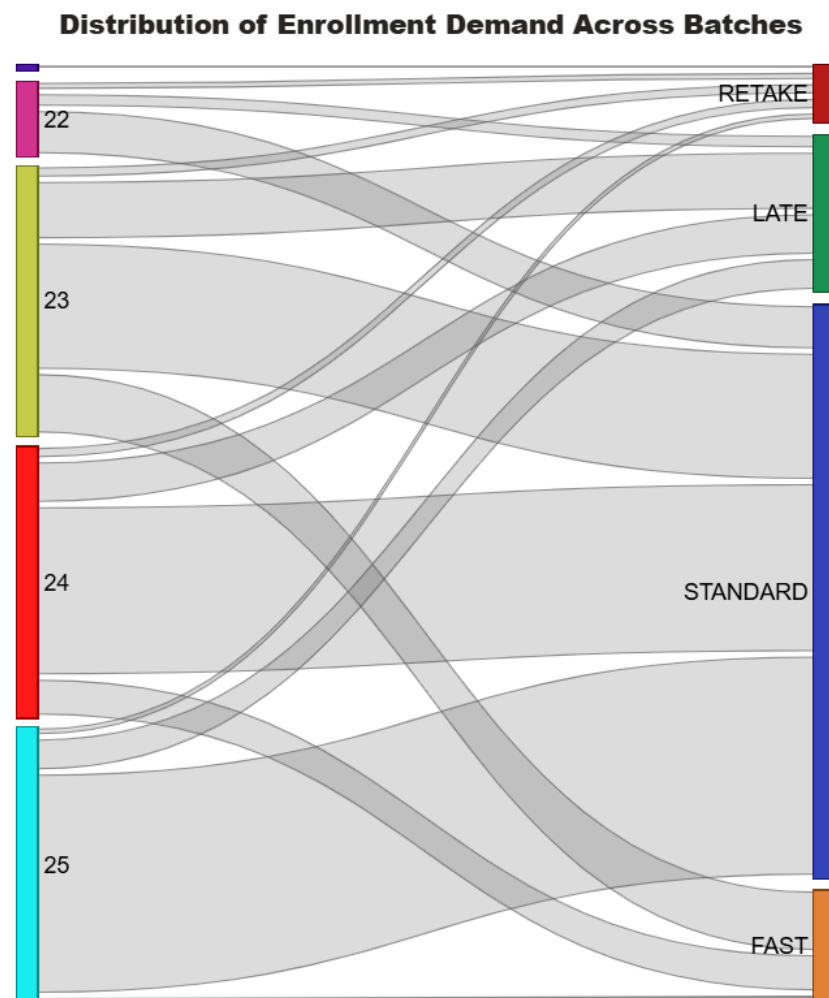


Figure 4.3: Distribution of Enrollment Demand Flow across Student Batches.

- **Visual Semantics:**
 - **Nodes:** The left vertical bar segments students by intake year (Batch 22 to Batch 25). The right vertical bar represents the four demand types (Retake, Late, Standard, Fast).

- **Links:** The width of the connecting bands is proportional to the volume of course registrations. A wider band indicates a stronger correlation between a specific batch and a demand type.
- **Key Analytical Patterns:**
 - **The "Standard" Dominance in Junior Years:** Batch 25 (Freshmen/-Cyan) and Batch 24 (Sophomores/Red) exhibit the thickest flows towards the **Standard (Blue)** node. This confirms that in the early years of the program, students largely adhere to the rigid structure of the curriculum roadmap.
 - **Accumulation of "Academic Debt":** As we observe older cohorts, specifically Batch 22 (Seniors/Magenta) and Batch 23 (Juniors/Yellow), the flows diverge significantly. The proportion of connections to **LATE (Green)** and **RETAKE (Red)** increases noticeably relative to their total size. This visually validates the necessity of the complex "Late Demand" simulation logic (Phase 3), as seniors are actively clearing backlog courses to graduate.
 - **Acceleration Trends:** The **Fast (Orange)** demand is fed by multiple streams, but notably from Batch 25. This suggests that a segment of new students is utilizing English proficiency exemptions or high entry scores to register for courses ahead of their standard schedule immediately upon entry.
- **Strategic Implication:** This chart prevents "one-size-fits-all" planning. It demonstrates that scheduling for Freshmen requires rigid block scheduling (Standard), whereas scheduling for Seniors requires flexible slots to accommodate the high volume of disparate "Late" and "Retake" module registrations.

Risk Assessment: Course Risk & Progression Matrix (Bubble Chart)

To identify critical bottlenecks and courses requiring academic intervention, the dashboard utilizes a multi-dimensional Bubble Chart (Figure 4.4). This visualization correlates failure rates with irregular curriculum progression.

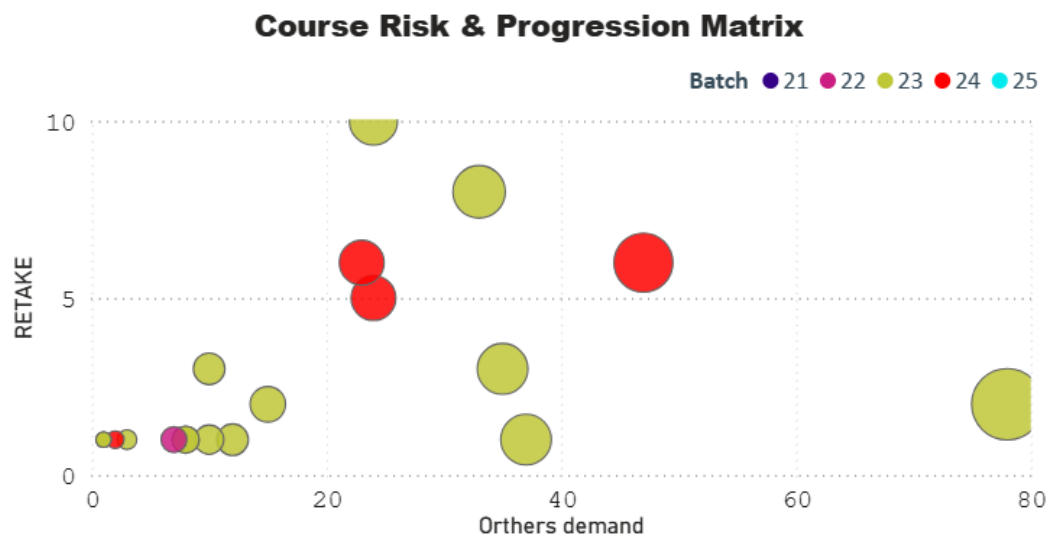


Figure 4.4: Course Risk Matrix correlating Retake Demand (Y-axis) against Other Demand (X-axis).

- **Visual Semantics:**

- **Axes:** The Y-axis represents **Retake Demand** (Number of failing students). The X-axis represents **"Others" Demand** (Aggregation of Late and Fast demand).
- **Bubbles:** Each bubble represents a specific course-batch pairing.
- **Size & Color:** The bubble size indicates total enrollment volume, while the color differentiates the student cohort (e.g., Yellow for Batch 23, Red for Batch 24).

- **Analytical Insights:**

- **High-Risk Clusters (Batch 23 - Yellow):** The visualization reveals that Batch 23 (Juniors) currently faces the highest volatility. There is a distinct cluster of yellow bubbles in the upper-left quadrant (High Retake, Low Other), indicating "Gatekeeper Courses" where many students are failing but few are taking them out of sequence.
- **The "Bottleneck" Anomaly:** A significant outlier appears on the far right (Yellow bubble at $X \approx 80, Y \approx 2$). This represents a course with massive "Backlog/Late" demand but very low failure rates. This suggests a scheduling conflict in previous semesters caused a mass delay in registration, rather than the course being academically difficult.
- **Emerging Risks in Batch 24 (Red):** While Batch 24 is younger, they show several bubbles climbing the Y-axis ($Retake > 5$). Early detection of these courses allows the faculty to deploy tutoring resources before these students reach their final year.

- **Strategic Implication:** This matrix allows the Academic Affairs Office to triage resources:

- **High Y-axis:** Requires *Academic Support* (Tutors, revision classes).
- **High X-axis:** Requires *Logistical Support* (Opening more class sections to clear the backlog).

Comparative Cohort Dynamics (Ribbon Chart)

To analyze how the "Demand Mix" shifts as students mature through the program, the dashboard employs a Ribbon Chart (Figure 4.5). This visualization is unique in its ability to show both the total volume (height of the bar) and the relative ranking of each demand type within a cohort.

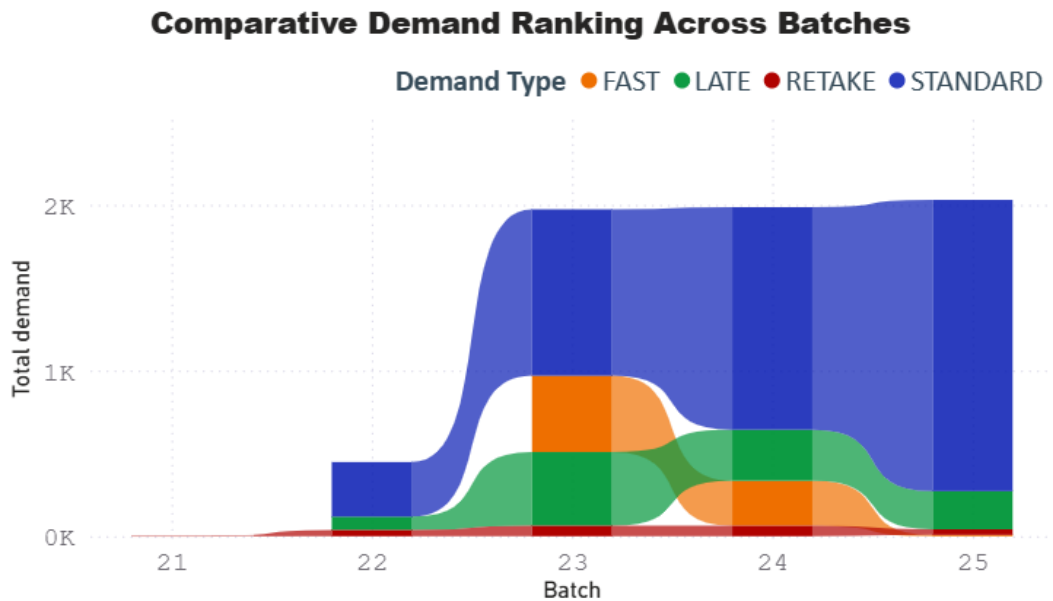


Figure 4.5: Comparative Demand Ranking and Composition Across Batches.

- **Visual Semantics:**

- **X-Axis (Cohorts):** Represents student batches from oldest (Batch 21) to youngest (Batch 25).
- **Y-Axis (Volume):** Represents the total registered credit volume.
- **Ribbons:** The colored bands represent the demand types. Their vertical ordering within a column indicates their relative dominance for that specific batch (e.g., the largest category is at the top).

- **Analytical Insights:**

- **The "Standard" Baseline (Blue):** For the younger cohorts, specifically Batch 25 (Freshmen) and Batch 24 (Sophomores), the **Standard Demand** is overwhelmingly dominant, occupying the largest surface area. This confirms that early-year scheduling should prioritize block-based, standard timetabling.
- **The "Acceleration" Surge (Batch 23 - Orange):** A striking anomaly is observed in Batch 23 (Juniors). The **Fast Demand (Orange)** ribbon expands significantly here compared to other years. This suggests a specific curriculum characteristic in the third year that opens up opportunities for electives or advanced modules, which students are aggressively utilizing to speed up their graduation.
- **The "Debt" Accumulation (Batch 24 - Green):** While Batch 24 has high Standard demand, the **Late Demand (Green)** ribbon is notably thicker than in Batch 25. This signals the "Sophomore Slump" phenomenon, where students begin to accumulate backlog courses from their first year, creating early scheduling conflicts.
- **Senior Year Attrition (Batch 21/22):** The total volume drops sharply for the oldest batches, which is expected as students graduate. However, the composition shifts: the ratio of **Retake (Red)** and **Late (Green)** to

Standard becomes much higher, indicating that the remaining students are primarily struggling to clear final requirements.

- **Strategic Implication:**

- **For Batch 25:** Focus on optimizing large lecture halls (Standard).
- **For Batch 23:** Focus on resolving conflicts between Year 3 Standard courses and Year 4 Advanced courses (Fast).
- **For Batch 21/22:** Prioritize "Rescue Courses" – opening specific sections for high-failure subjects to ensure these students can graduate.

Capacity Planning: Space Allocation Strategy (Treemap)

To translate the abstract demand figures into physical infrastructure requirements, the dashboard utilizes a Treemap visualization (Figure 4.6). This chart specifically filters for courses requiring **more than 50 seats**, effectively isolating the high-volume modules that dictate the master schedule for large lecture halls.

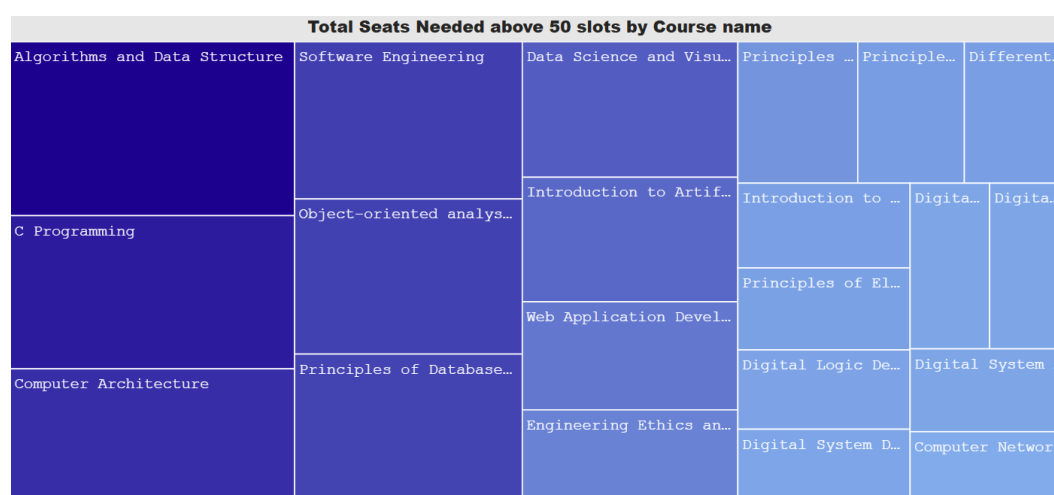


Figure 4.6: Total Seats Needed (Courses > 50 slots) visualized by Volume and Intensity.

- **Visual Semantics:**

- **Area Size:** The surface area of each rectangle is directly proportional to the total number of seats required. This allows for an instant visual comparison of "spatial footprint" between courses.
- **Color Gradient:** The color intensity (from light blue to dark blue) reinforces the volume metric—darker blocks indicate the highest density of students, demanding the largest resources.

- **Analytical Insights:**

- **The "Auditorium Tier" (Dark Blue):** The visualization is dominated by three massive blocks: "Algorithms and Data Structures", "C Programming", and "Computer Architecture". These courses represent the "Facility Anchors." They must be scheduled first, as they are the only ones requiring the university's largest auditoriums (capacity > 150). Any conflict in scheduling these three would result in a logistical deadlock.

- **The "Mid-Range Tier" (Medium Blue):** Courses like *"Software Engineering"* and *"Object-Oriented Analysis"* form the second tier. While they don't require the main hall, they exceed the capacity of standard classrooms. They compete for the limited number of medium-sized lecture rooms (capacity 80–120).
- **The "Long Tail" of Standard Rooms (Light Blue):** The smaller, lighter rectangles on the right (e.g., *"Digital Logic"*, *"Engineering Ethics"*) represent courses that can fit into standard classrooms. Although numerous, they offer high scheduling flexibility compared to the darker blocks.
- **Operational Value:** This visualization serves as a direct input for the *Room Assignment Algorithm*. It visually enforces a "Big Rocks First" strategy: schedule the dark blue blocks into the scarcest rooms first, then fit the lighter blue blocks into the remaining spaces.

Comprehensive Demand Composition (Vertical Stacked Bar Chart)

While the previous charts focused on high-level trends or top-tier logistics, the "Student Registration Demand Breakdown" (Figure 4.7) provides a forensic analysis of the entire course catalog. This visualization is essential for handling the "Long Tail" of the curriculum.

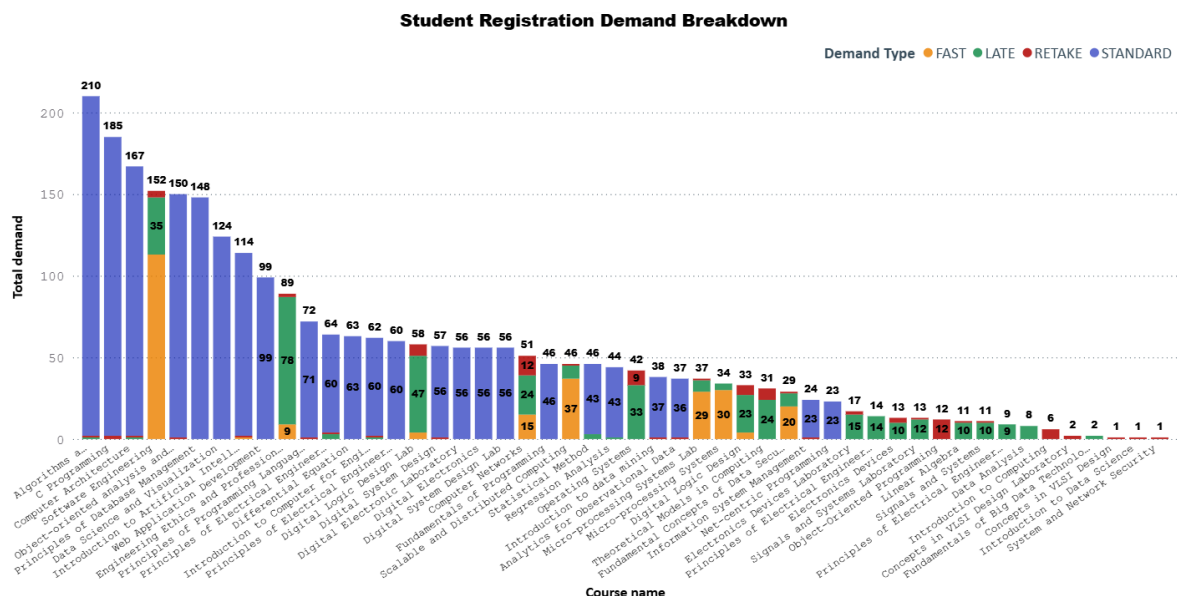


Figure 4.7: Full Breakdown of Course Demand by Type (Standard, Late, Fast, Retake).

- **Visual Semantics:**
 - **Ranking:** Courses are sorted in descending order of total demand volume from left to right.
 - **Composition:** Each bar is vertically stacked with color-coded segments, allowing for an immediate visual ratio analysis of *Why* students are taking the course.
- **Analytical Insights & Forensics:**

- **The "Standard" Giants (Left Side):** The chart confirms that the heaviest logistical load comes from fundamental courses like "*Algorithms and Data Structures*" (210) and "*C Programming*" (185). These bars are overwhelmingly **Blue (Standard)**, indicating stable, predictable demand that fits the standard timetable blocks.
 - **The "Software Engineering" Anomaly:** This course serves as a critical outlier. despite being a high-volume course (152 students), it has almost **zero Standard demand**. It is composed entirely of **Fast (Orange)** and **Late (Green)** segments.
 - * *Implication:* This indicates that no specific cohort is currently scheduled for this course in their roadmap. The demand is purely driven by juniors learning ahead and seniors clearing backlog. This requires a custom schedule that does not conflict with either Year 3 or Year 4 standard slots.
 - **The "Bottleneck" Courses (Green Dominance):** Courses such as "*Engineering Ethics*" (89) and "*Principles of Programming Languages*" (72) display a massive **Late (Green)** component relative to their size. This suggests these subjects are frequently delayed by students, perhaps due to low perceived priority or prerequisite issues in earlier semesters.
 - **The "Struggle" Tail (Right Side):** As we move to the lower-volume courses on the right (e.g., "*Linear Algebra*", "*Data Analysis*"), the **Retake (Red)** segments become proportionately much larger. For instance, *Linear Algebra* (12 students) appears to be almost entirely composed of retake students. These are likely special "rescue sections" needed for graduation.
- **Operational Value:** This chart drives the "Sectioning Strategy":
 - **Blue-heavy bars:** Open standard sections restricted to specific batches.
 - **Mixed (Orange/Green) bars:** Open "Free Elective" sections accessible to all years.
 - **Red-heavy bars:** Open small, intensive remedial classes.

Chapter 5

RESULT

5.1 Experimental Dataset Description

To validate the proposed system’s functionality and performance, a comprehensive synthetic dataset was generated. This dataset simulates the academic environment of the International University (IU) for the academic year 2025-2026, serving as the ground truth for evaluating the course demand forecasting model.

5.1.1 Dataset Summary

The resulting dataset comprises historical academic records spanning from 2017 to 2025, with a total student population of 1,197.

The student population distribution follows a realistic growth trend, with newer cohorts having higher enrollment numbers to reflect recent intake adjustments. The detailed distribution by cohort is described below:

- **Delayed Cohorts (K17–K20):** A total of 52 students who have not yet graduated due to academic delays.
- **Active Cohorts (K21–K24):** The main body of students, totaling 898, currently following the standard curriculum.
- **New Intake (K25):** 247 freshmen entering the forecast period.

Cohort	Students	Cohort	Students
K17	2	K22	228
K18	9	K23	222
K19	19	K24	231
K20	22	K25	247
K21	217	Total	1,197

5.1.2 General Dashboard Overview

General Overview Page

The General Dashboard Overview (Figure 5.1) serves as the executive landing page for the Academic Affairs Office. It is designed to answer the fundamental question: *“What is the overall scale and structure of the upcoming semester’s demand?”*

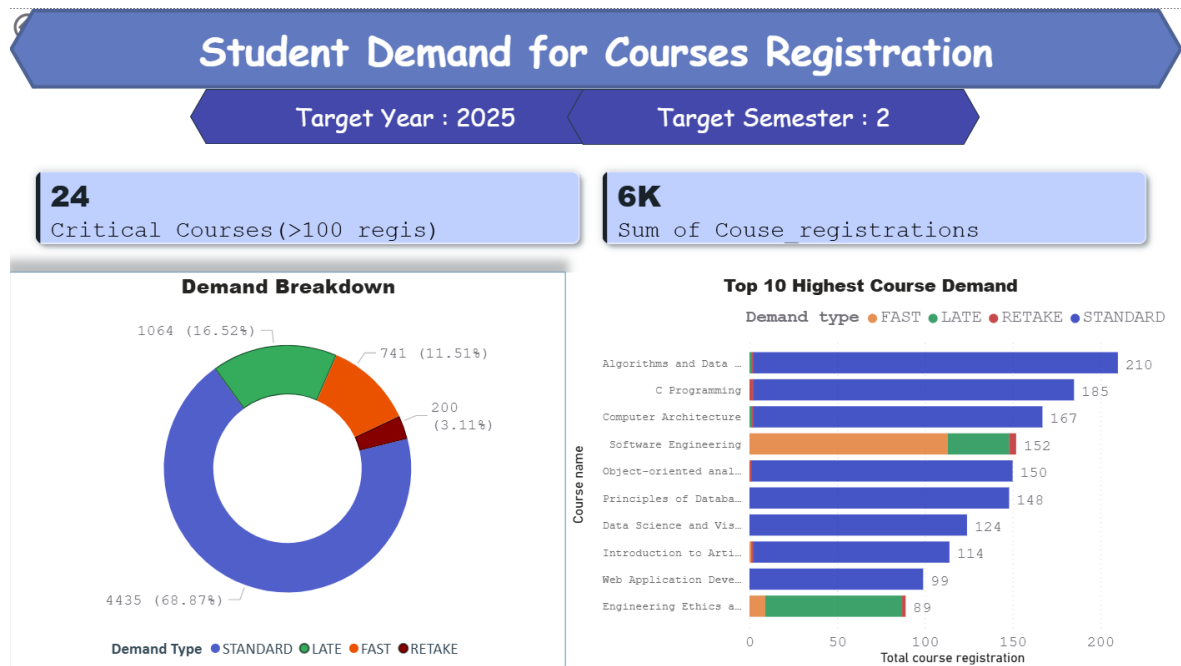


Figure 5.1: General Dashboard Overview: Key Metrics and High-Level Demand Distribution.

The "Demand Breakdown" chart on the left provides a strategic health check of the student body's progression. It aggregates the total forecasted registrations (approx. 6,000 seats) into the four distinct logic categories. Two key insights are immediately visible:

- **Demand Structure:** The Donut Chart reveals that while **Standard Demand** (Blue) is dominant at 68.87%, nearly one-third of the demand originates from irregular sources (Late, Fast, Retake). This signals that a standard scheduling template will fail to accommodate approximately 30% of the student body.
- **Resource Anchors:** The Stacked Bar Chart identifies the "Heavyweight" courses. *Algorithms and Data Structures* and *C Programming* lead the list, driven primarily by standard cohorts (Blue bars), indicating the need for large lecture halls.

The "Top 10 Highest Course Demand" chart on the right ranks the courses with the highest projected enrollment numbers. Crucially, it uses a stacked bar format to show the *composition* of that demand, allowing administrators to distinguish between standard mass courses (e.g., *Algorithms*) and complex bottleneck courses (e.g., *Software Engineering*).

A critical feature of the dashboard is its **Cross-Filtering capability**. The static view only tells half the story; the interactive nature allows decision-makers to isolate specific problems.

For instance, by clicking on the green "Late Demand" slice in the Donut Chart, the entire dashboard contextually filters to focus solely on the 1,064 students with backlog issues (as illustrated in Figure ??).

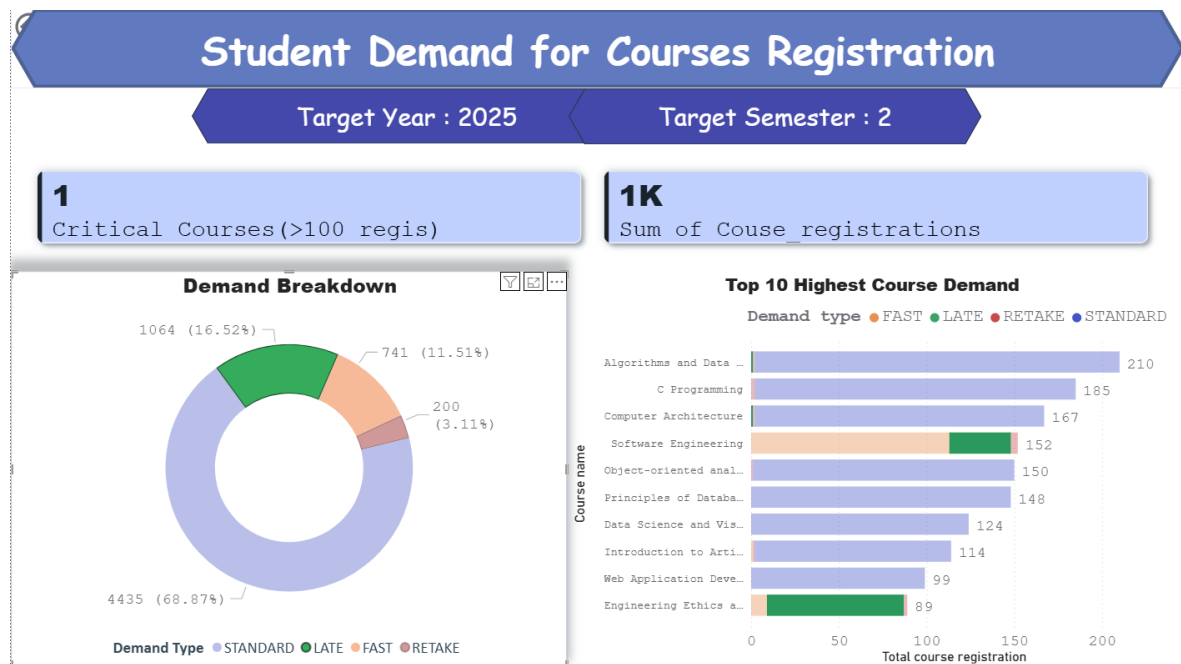


Figure 5.2: Interactive Scenario: The dashboard filtered by 'Late Demand'. Note the shift in the 'Top 10' chart highlighting backlog bottlenecks.

Upon this interaction, the dashboard performs a **Cross-Filtering** operation, yielding specific insights:

- **Volume Isolation:** The KPI card updates immediately, showing that "Late Demand" accounts for exactly **1,064 course registrations**.
- **Bottleneck Identification:** The "Top 10" chart drastically changes.
 - The blue bars for standard courses (e.g., *Algorithms*) fade out, confirming they are not the primary source of backlog issues.
 - Conversely, the green segments in *Software Engineering* and *Engineering Ethics* remain highlighted and dominant.
- **Actionable Intelligence:** This visual feedback proves that "Late" demand is not evenly distributed. It is concentrated in specific "Gatekeeper Courses." Consequently, the faculty knows exactly which subjects require extra remedial sections, rather than randomly increasing capacity across the board.

Cohort Dynamics and Risk Assessment Page

While the General Overview provides a snapshot of total volume, the **Cohort Dynamics Page** (Figure 5.3) offers a deep dive into student behavior across different academic years. This view is essential for the Academic Advisors to tailor their support strategies for specific student batches (e.g., Freshmen vs. Seniors).

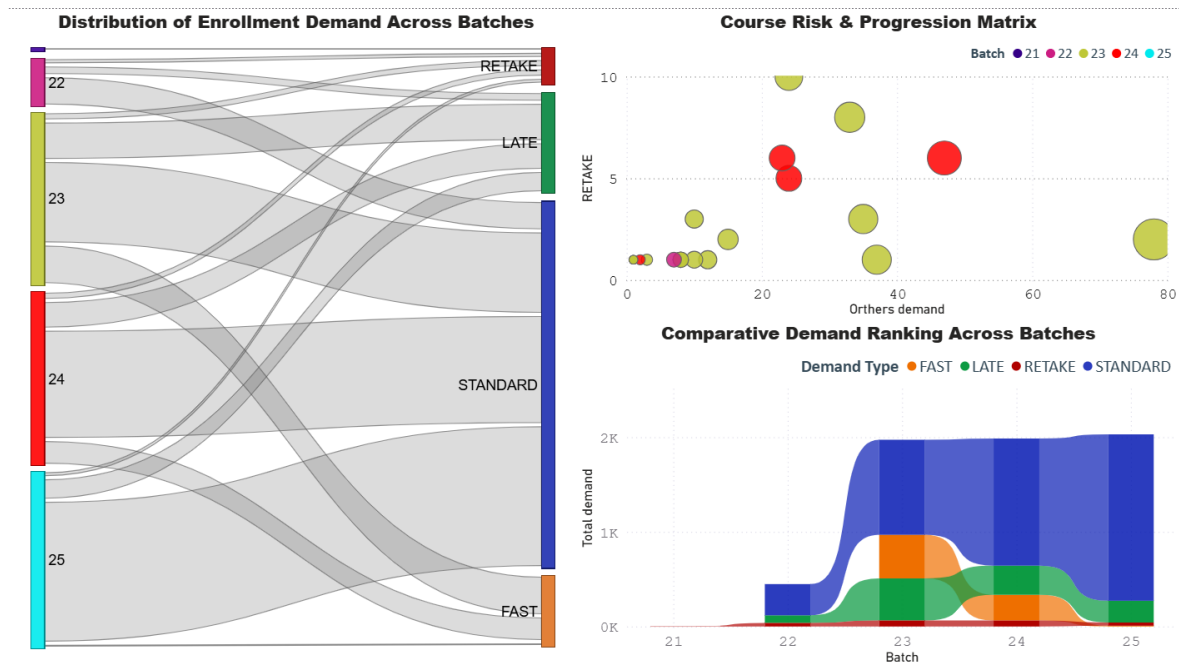


Figure 5.3: Cohort Dynamics Dashboard: analyzing demand flow, risk factors, and volume evolution across student batches.

The "Distribution of Enrollment Demand Across Batches" chart maps the relationship between student cohorts (Batches 22–25) and their requested demand types.

- **Function:** It visualizes the complexity of course registration. Thicker bands indicate higher volume.
- **Key Insight:** It highlights a behavioral shift.
 - **Batch 25 (Freshmen - Cyan):** The flow is almost entirely directed towards **Standard** demand, indicating high curriculum adherence.
 - **Batch 22 (Seniors - Magenta):** The flows fragment significantly into **Late** (Green) and **Retake** (Red), visualizing the "Cleanup Phase" where seniors scramble to clear backlog and failed courses.

The "Course Risk & Progression Matrix" is a diagnostic tool correlating academic failure with irregular curriculum progression.

- **Function:**
 - **Y-Axis (Retake):** Represents academic difficulty (High failure count).
 - **X-Axis (Others Demand):** Represents logistical irregularity (High volume of Late/Fast registrations).
 - **Color:** Represents the specific Batch (e.g., Yellow = Batch 23).
- **Key Insight:** The chart identifies **Batch 23 (Juniors - Yellow)** as the most critical group. They appear frequently in the upper quadrants, indicating they are facing a "Junior Slump"—simultaneously struggling with difficult courses (High Y) and managing complex backlog schedules (High X).

The "Comparative Demand Ranking" chart visualizes how the composition of demand evolves as the student body matures from Batch 25 to Batch 21.

- **Function:** It stacks the demand volume to show both total magnitude and the ratio of demand types.
- **Key Insight:**
 - **The Acceleration Window:** The **Fast Demand (Orange)** section expands significantly at Batch 23, identifying the Third Year as the prime window where high-performing students attempt to learn ahead.
 - **The Debt Accumulation:** The **Late Demand (Green)** begins to widen starting from Batch 24, visually proving that backlog accumulation begins as early as the sophomore year.

A key capability of the system is its interactive filtering, allowing administrators to isolate specific cohorts or demand types to diagnose root causes. Two specific scenarios are presented below:

By selecting the **Batch 23** node on the Sankey Diagram, the entire dashboard filters to display only the data relevant to the Junior cohort (Figure 5.4).

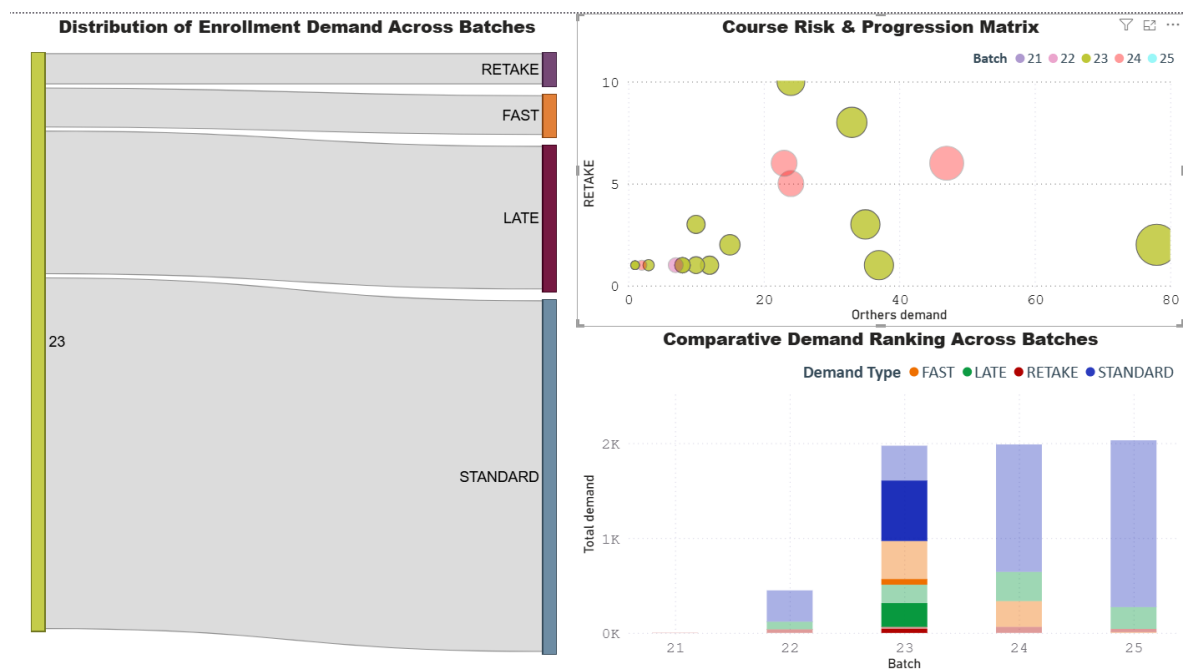


Figure 5.4: Drill-Down: Dashboard filtered by 'Batch 23' (Juniors). The Ribbon Chart highlights the 'Acceleration Window' (Orange Segment).

- **Observation:** The filtering reveals that Batch 23 is the most complex group. The Ribbon Chart (bottom right) shows that this cohort has the widest **Orange Band (Fast Demand)** compared to any other year.
- **Insight:** This confirms that the third year is the primary "Acceleration Window," where students aggressively utilize electives to speed up graduation. Simultaneously, the Bubble Chart isolates the yellow bubbles, showing a high variance in risk—some students are failing significantly while others are accelerating.

By selecting the **LATE** node on the right side of the Sankey Diagram, the dashboard performs a reverse-lookup to identify which batches are contributing to the backlog (Figure 5.5).

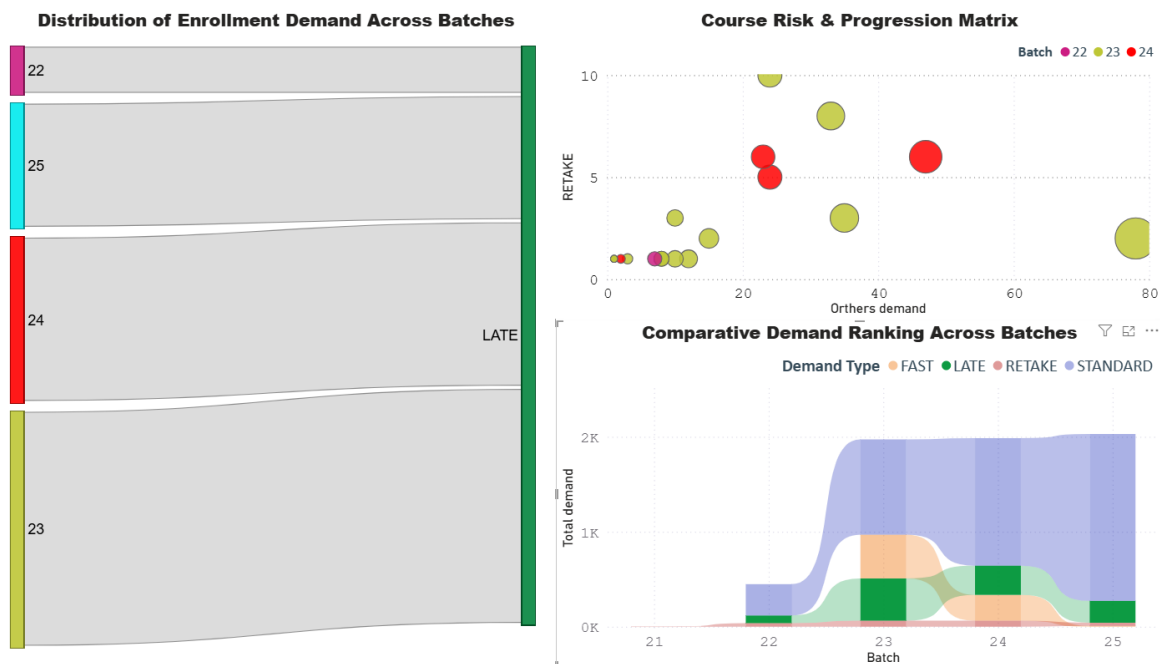


Figure 5.5: Drill-Down: Dashboard filtered by 'Late Demand'. The Sankey lines reveal that backlog issues originate largely from Batches 23 and 24.

- **Observation:** The Sankey flow highlights that "Late Demand" is not exclusive to Seniors. Significant green streams originate from **Batch 24 (Sophomores)** and **Batch 23 (Juniors)**.
- **Insight:** This contradicts the assumption that only seniors have debt. It proves that "Academic Debt" begins accumulating as early as the second year (Sophomore Slump). This insight allows the faculty to intervene early with Batch 24 to prevent them from becoming the high-risk seniors of Batch 22 in the future.

Detailed Scheduling and Infrastructure Planning Page

The next dashboard view (Figure 5.6) serves as the operational hub for the scheduling committee. While the previous pages identified trends and risks, this page provides the granular data required to physically construct the semester timetable and allocate classroom resources.

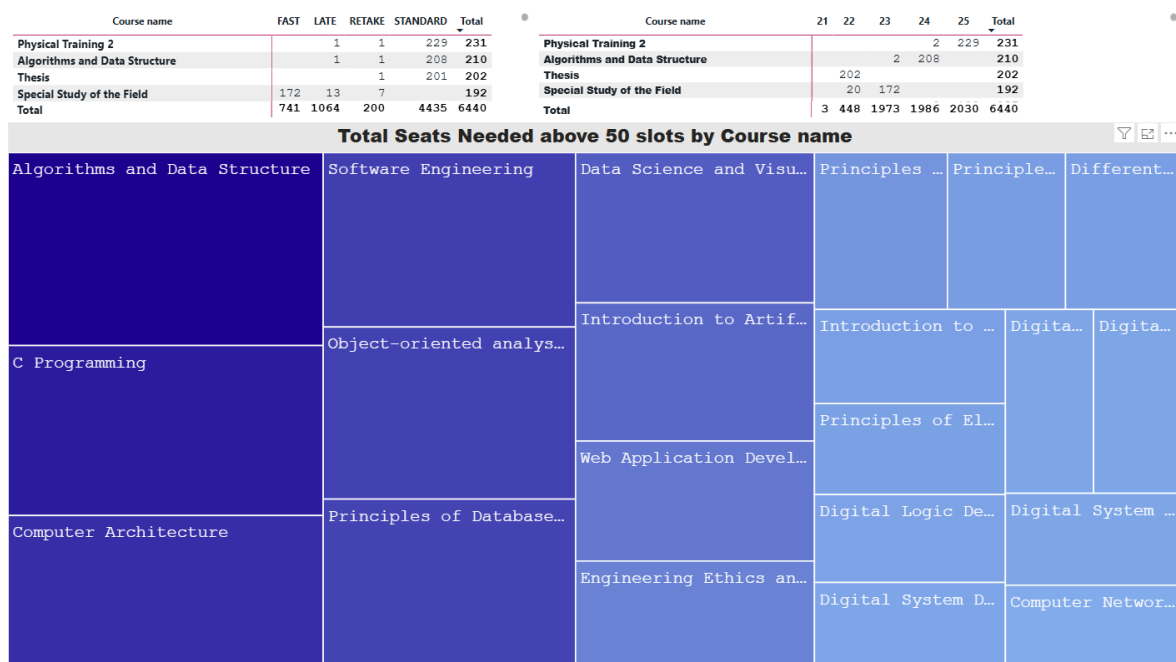


Figure 5.6: Infrastructure Planning Dashboard: Granular Demand Matrices and Seat Capacity Treemap.

The upper section features two detailed cross-tabulation tables that act as the "Master List" for course registration.

- **Demand Composition Matrix (Left):** Breakdowns total demand by type (Fast/Late/Retake/Standard).
 - *Insight:* It reveals specific anomalies like "Special Study of the Field", where **172 out of 192** registrations are "Fast Demand." This tells the scheduler that this course is not driven by the standard curriculum but by high-performing students accelerating their studies.
- **Cohort Distribution Matrix (Right):** Breakdowns total demand by Batch.
 - *Insight:* It assists in conflict avoidance. For example, "Thesis" is populated exclusively by **Batch 21 and 22**, whereas "Physical Training 2" is almost entirely **Batch 25**. This confirms that these two courses can be scheduled in overlapping time slots without causing conflicts.

The "Total Seats Needed above 50 slots" Treemap translates the abstract numbers into physical spatial requirements. By filtering out small classes (<50), it focuses attention on the "Big Rocks"—the large courses that dictate the master schedule.

- **Visual Hierarchy:**
 - **Area Size:** Represents the physical footprint required.
 - **Color Intensity:** Darker blue indicates higher student density.
- **Resource Triage:**
 - **Tier 1 (Dark Blue):** Courses like *Algorithms and Data Structures* and *C Programming* occupy the largest, darkest blocks. These must be assigned first to the university's limited Auditorium capacity.

- **Tier 2 (Medium Blue):** Courses like *Data Science* and *Software Engineering* require large lecture halls (100+ capacity).
- **Tier 3 (Light Blue):** Smaller blocks like *Digital System Design* can be flexibly assigned to standard classrooms.

This page supports a specific decision-making workflow:

1. **Identify Volume:** Look at the Treemap to determine which courses need the Auditoriums.
2. **Check Composition:** Consult the Top Left Matrix to see if the demand is Standard (predictable) or Late/Retake (requires special evening/weekend slots).
3. **Check Conflict:** Consult the Top Right Matrix to ensure the chosen time slot does not conflict with other courses taken by the same Batch.

A key feature of this page is the ability to select a specific course in the Treemap to audit its exact composition. Figure 5.7 illustrates the result of selecting the "C Programming" block.

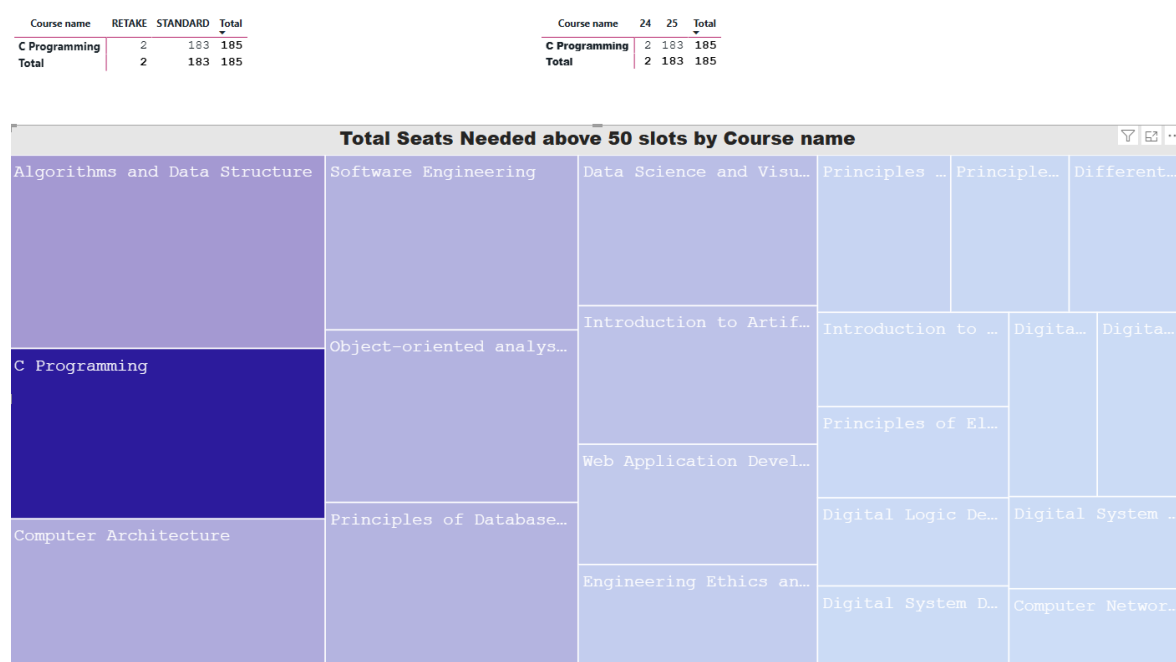


Figure 5.7: Interactive Scenario: Selecting 'C Programming' reveals it is a pure Freshman/Sophomore course with minimal irregular demand.

Upon selection, the two upper matrices filter contextually to reveal the profile of this specific course:

- **From the Left Matrix (Demand Type):** We observe that out of 185 total seats, **183 are Standard** and only **2 are Retake**.
 - *Operational Decision:* This confirms that "C Programming" does not require special remedial sections. It can be scheduled as a standard mass lecture.

- **From the Right Matrix (Cohort):** The data shows the population is dominated by **Batch 25 (183 students)** with a minor presence from **Batch 24 (2 students)**.
 - *Operational Decision:* The scheduler now knows to treat this as a "Freshman Block." They must ensure this course does not conflict with other Batch 25 compulsory subjects (like *Calculus 1* or *Physical Training*), but they do not need to worry about conflicts with Senior year courses.

Comprehensive Demand Composition and Filtering Page

The final component of the dashboard (Figure 5.8) is the "Student Registration Demand Breakdown." This page is designed for Forensic Data Analysis. It allows the Academic Affairs Office to answer specific, ad-hoc questions that the high-level overviews might miss (e.g., "Which courses are K23 students accelerating?" or "What is the exact volume of Retake demand for Calculus?").

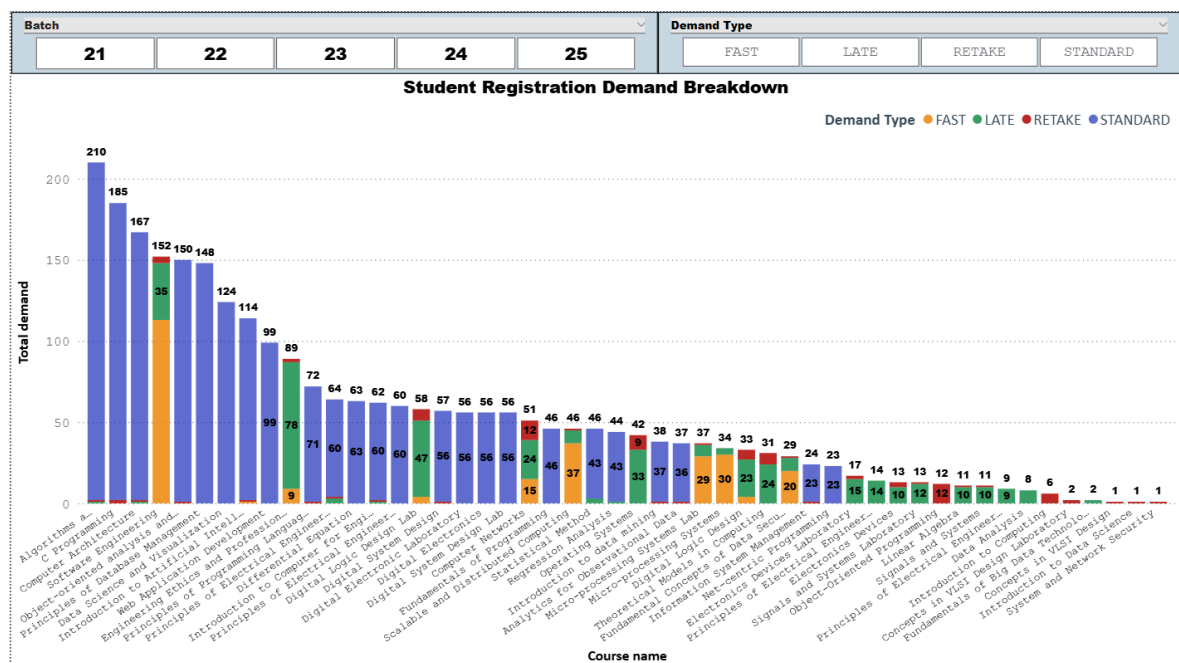


Figure 5.8: Comprehensive Demand Breakdown (Default View): Showing all courses sorted by total volume with color-coded demand composition.

Administrators can isolate the demand profile of a single student cohort. Figure 5.9 illustrates the result of selecting "Batch 24".

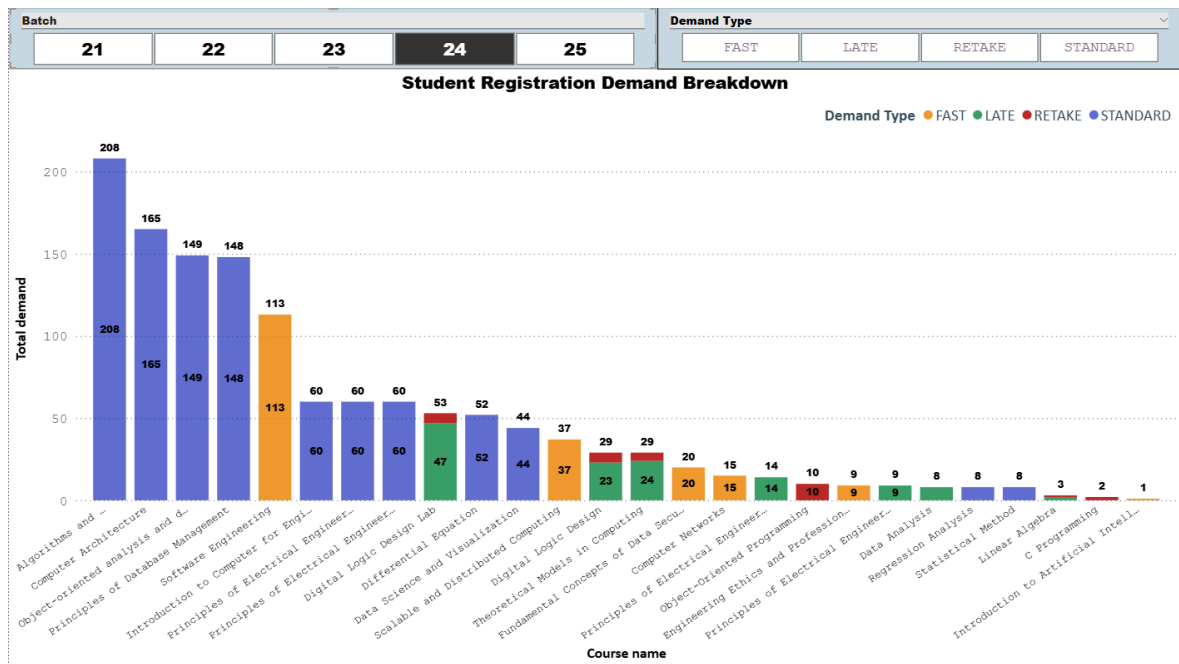


Figure 5.9: Interactive Scenario: Filtering by 'Batch 24'

- **Visual Shift:** The chart re-ranks instantly. Courses irrelevant to Batch 24 disappear.
- **Insight:** We observe a dual-behavior in this cohort:
 - **Standard Load:** The tallest blue bars (e.g., *Algorithms*, *Computer Architecture*) represent their main curriculum.
 - **Acceleration:** The prominent orange bar for *Software Engineering* (113 students) confirms that a large segment of this batch is choosing to study this subject ahead of schedule.

To analyze specific academic behaviors, users can filter by Demand Type. Figure 5.10 shows the dashboard filtered specifically for "FAST" demand.



Figure 5.10: Interactive Scenario: Filtering by 'FAST' demand

- **Function:** This removes all "Standard" noise, leaving only the acceleration requests.
- **Insight:** It becomes immediately clear that *Software Engineering* and *Scalable Distributed Computing* are the primary targets for high-performing students. This data is vital for deciding whether to open advanced elective sections.

The system supports Combined Filtering (e.g., selecting "Batch 24" AND "LATE").

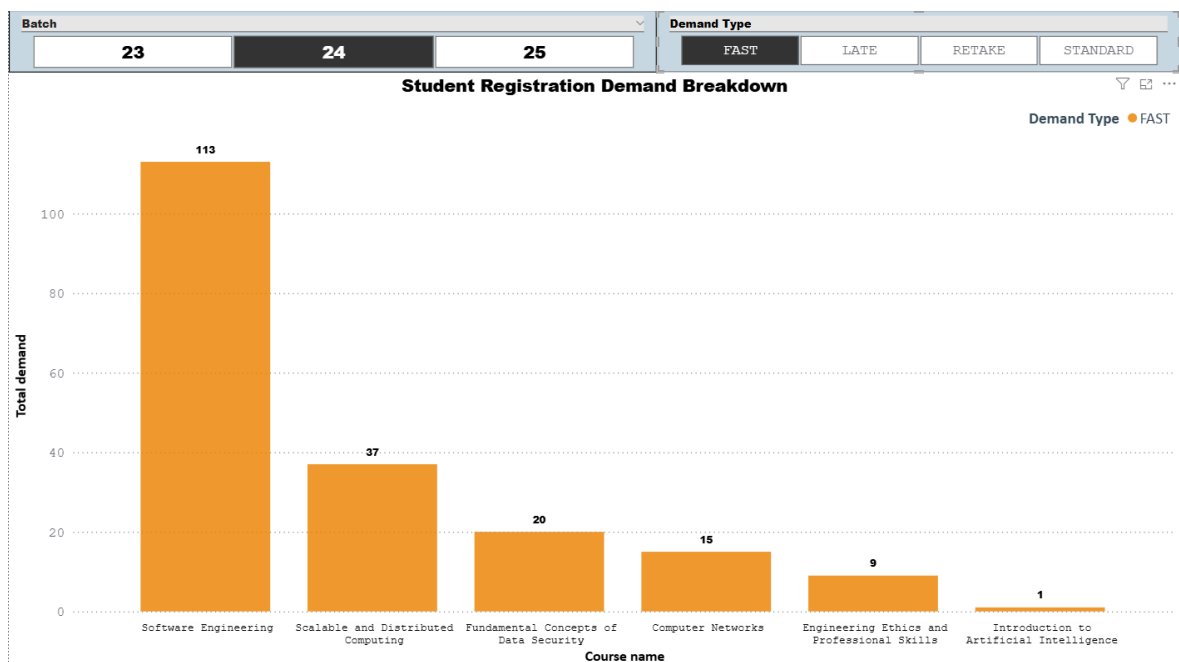


Figure 5.11: Interactive Scenario: Combining filtering by 'FAST' demand and batch 24

- *Example:* By combining filters, an administrator can generate a list of exactly which Year 2 students (Batch 24) are retaking Year 1 Math courses, enabling targeted academic advising interventions.

5.1.3 Summary of Experimental Results

The experimental evaluation confirms that the proposed data-driven system successfully processed, classified, and visualized the complex course registration demand for Semester 1, Academic Year 2025-2026. The integration of the SQL estimation engine with the Power BI analytics layer provided a multi-dimensional view of student behavior, yielding the following critical observations:

- **Quantification of Irregular Demand:** The analysis reveals that a rigid, curriculum-based estimation model would fail to capture nearly one-third of the actual demand. While **Standard Demand** accounts for **68.87%** (4,435 registrations), the aggregate of **Irregular Demand** (Late, Fast, Retake) constitutes a significant **31.13%** (2,005 registrations). This highlights the necessity of the constraint-based simulation logic (Phases 3 and 4) implemented in this study.
- **Identification of Structural Bottlenecks:** The dashboard successfully differentiated between "Volume Anchors" and "Logistical Bottlenecks."
 - Courses like *Algorithms and Data Structures* require large auditoriums due to mass standard enrollment.
 - Conversely, courses like *Software Engineering* were identified as "Hidden Bottlenecks," driven almost entirely by Fast (Batch 23) and Late (Batch 22) demand, requiring flexible scheduling outside standard time slots.
- **Detection of Early-Onset Academic Risk:** The Cohort Analysis debunked the assumption that "Academic Debt" is an issue exclusive to seniors. The data showed significant "Late Demand" accumulation beginning as early as **Batch 24 (Sophomores)**, particularly in foundational programming and math courses. This insight enables the faculty to deploy early interventions before these students reach their final year.
- **Operational Feasibility:** The visualization layer proved effective in supporting diverse decision-making workflows, from high-level strategic resource planning (Dean's view) to granular conflict checking for individual course sections (Scheduler's view).

In conclusion, the system transforms the course planning process from a reactive, estimation-based task into a proactive, data-driven strategy, ensuring that infrastructure allocation aligns precisely with the diverse needs of the student body.

Chapter 6

DISCUSSION

6.1 Analysis of Results

The most significant finding is the quantification of Irregular Demand. While the curriculum roadmap accounts for **68.87%** of registrations (Standard), a substantial **31.13%** of demand arises from deviations (Retake, Late, Fast). This confirms that a rigid, roadmap-based estimation model would fail to capture nearly one-third of the actual student needs. Several critical observations can be drawn from this data:

6.1.1 The "Ripple Effect" of Prerequisite Chains

The analysis of the **Risk Matrix (Bubble Chart)** and **Retake Demand** reveals that the curriculum structure creates a significant "Ripple Effect," where a failure in one course causes downstream disruptions.

- **Bottleneck Identification:** Courses such as *Data Structures & Algorithms (IT013)* and *Object-Oriented Programming (IT069)* act as "Gatekeepers." High failure rates in these subjects do not only generate immediate Retake Demand but effectively block access to subsequent courses like *Web Development* or *Artificial Intelligence*.
- **Demand Transformation:** The **Sankey Diagram** visually confirms the shift from "Standard" to "Late." A student failing a prerequisite in Semester 3 causes a void in Standard Demand for Semester 4, followed by a surge in "Late Demand" in Semesters 5 or 6. This insight allows the faculty to predict "delayed waves" of enrollment, ensuring that elective courses are not opened when the eligible student pool is still stuck in prerequisite bottlenecks.

6.1.2 Structural Delays: The English Barrier

The "Late Demand" analysis validates the unique characteristic of the International University environment regarding English proficiency.

- **The English Barrier Effect:** A significant portion of the **K24 cohort (Sophomores)** was classified as "Late" for Year 1 major courses. This validates the simulation logic (Phase 3), which correctly identified that these students were blocked from registering for major courses in their Freshman year due to entry-level English requirements (IE0/IE1).
- **Managerial Implication:** This disproves the assumption that "Late Demand" is solely caused by academic failure. Instead, it highlights a structural delay. The Academic Affairs Office must prepare for a "Sophomore Slump"—a delayed peak in demand for introductory courses in the second year—to accommodate students clearing their English preparatory phase.

6.1.3 Unveiling Latent Acceleration Demand

One of the most distinct findings, highlighted by the **Batch 23 data in the Ribbon Chart**, is the behavior of high-performing students.

- **The "Software Engineering" Anomaly:** As observed in the breakdown charts, courses like *Software Engineering* showed almost zero "Standard" demand but massive "Fast" demand. This indicates that juniors are aggressively utilizing their electives to take senior-level courses early.
- **Resource Optimization:** Without the "Fast Demand" simulation (Phase 4), these seats would remain unallocated (hidden capacity). Identifying this demand allows the university to fill empty slots in advanced classes with talented juniors, optimizing lecturer utilization.

6.1.4 Simulation Validity: Credit Constraints

The application of the Greedy Algorithm for credit limits proved to be the decisive factor in ensuring realistic forecasting.

- **Forced Prioritization:** In reality, students cannot take every course they need. By implementing the priority logic (*Retake, Late, Standard, Fast*), the system mimics the decision-making process of a student under the 24-credit constraint.
- **Preventing Overestimation:** The results demonstrate that students with heavy "Retake" burdens are forced to postpone their "Standard" courses. Without this constraint simulation, the forecast would dangerously overestimate the number of seats needed for standard courses, leading to efficient room allocation for students who are actually ineligible to register due to credit caps.

6.2 Strengths of the Proposed System

The developed system offers a paradigm shift from traditional estimation methods. By moving away from aggregate historical averages, the system leverages a deterministic, constraint-based approach to deliver high-fidelity forecasting. The core strengths are consolidated into three key dimensions:

6.2.1 Precision via Constraint-Based Bottom-Up Forecasting

The most significant strength of the system is its transition from "Trend-Based" to "Rule-Based" forecasting. Instead of assuming a generic growth rate, the engine calculates demand for every single active student based on their unique academic history. Crucially, this calculation is governed by a rigorous Constraint Logic Layer:

- **Legal Validity Check:** Unlike basic projection models, this system filters demand through the university's complex regulation matrix. It accounts for *Prerequisite Chains* (preventing premature registration), *English Proficiency Barriers* (blocking major courses for IE0/IE1/... students), and most importantly, the **24-Credit Limit**. By applying a Greedy Algorithm to prioritize core subjects over electives, the system distinguishes between theoretical demand (what students want) and feasible demand (what they are legally allowed to register).

- **Granular Classification:** The system does not merely output a total count. It successfully segments demand into actionable categories (*Standard, Late, Fast, Retake*). This granularity allows the faculty to identify specific phenomena—such as the “Sophomore Slump” or “Latent Acceleration Demand”—that would remain invisible in a traditional aggregate report.

The integration of the calculation engine with the Power BI visualization layer transforms raw data into Strategic Situational Awareness.

From Estimation to Diagnosis: The system empowers administrators to move beyond simple capacity planning (“How many seats?”) to root-cause diagnosis (“Why is there a backlog?”). As demonstrated by the *Course Risk Matrix* and *Cohort Ribbon Charts*, the system can isolate specific “Bottleneck Courses” and “At-Risk Cohorts,” enabling targeted academic interventions rather than broad, inefficient resource expansions.

6.2.2 Privacy-Preserving Development Framework

Recognizing the sensitivity of educational records, the thesis contributes a robust Synthetic Data Generation Engine.

- **Statistical Realism without Exposure:** The engine generates datasets that mathematically mirror real-world student behaviors—capturing skewed grade distributions, failure probabilities, and irregular dropout rates—without exposing a single byte of Personally Identifiable Information (PII). This ensures that the system’s logic can be rigorously tested, optimized, and shared for peer review while maintaining strict compliance with data privacy standards.

6.3 Limitations

Despite its demonstrated efficacy, the system operates under certain constraints that must be acknowledged:

6.3.1 Synthetic Data Bias vs. Behavioral Nuance

While the data generator uses statistical distributions (Gaussian, Skewed) to mimic reality, it cannot fully capture the unpredictability of human behavior.

- **External Factors:** Real-world decisions are influenced by financial constraints, personal health issues, peer pressure, or lecturer preferences. These qualitative variables are currently simulated as random noise rather than causal factors.
- **Strategic Delays:** The system assumes students want to graduate as fast as possible. In reality, some students may intentionally delay a hard course to avoid a strict professor, a behavior the current algorithm does not model.

6.3.2 Deterministic Rule Sets

The classification logic assumes rational and optimal behavior. For example, it assumes a student will prioritize retaking a failed course immediately. However, students often deviate from this optimal path, choosing to focus on easier subjects to boost their GPA before returning to difficult retake courses. This deviation can lead to slight discrepancies between the “Predicted Optimal Demand” and “Actual Registration.”

6.3.3 Supply-Side Absence

The current system focuses exclusively on **Demand Estimation** (how many seats are needed). It does not account for **Supply Constraints** (availability of classrooms, time slots, or qualified lecturers). Therefore, while it tells the university *what* is needed, it does not automatically solve *how* to schedule it within limited physical resources.

6.4 Comparison with Traditional Methods

To quantitatively assess the improvement offered by the proposed system, Table 6.1 compares it against the traditional manual estimation workflow often used in academic administration.

Table 6.1: Comparison between Manual Estimation and the Proposed System

Feature	Manual / Excel-based Estimation	Proposed Logic-based System
Data Source	Relies on aggregate historical trends (e.g., "Last year we opened 5 classes, so we do the same this year").	Utilizes individual student records (Bottom-up approach), aggregating demand from specific student needs.
Prerequisite Enforcement	Often overlooked or estimated loosely due to the complexity of checking thousands of records manually.	Strictly enforced via Graph Logic/SQL Recursion, ensuring no ineligible student is counted.
Handling Delays	Difficult to track how many students are delayed due to English or prerequisites, leading to "surprise" shortages.	Automatically detects and classifies "Late Demand" based on curriculum mapping and student status.
Scalability	Low; prone to human error when handling datasets of thousands of students.	High; processes thousands of records efficiently using optimized SQL queries.

The comparison highlights that the proposed system provides a more accurate, transparent, and automated approach, reducing the administrative burden and minimizing human error in academic planning.

Chapter 7

CONCLUSION

7.1 Summary

This thesis addressed the critical challenge of academic resource planning at the International University (IU) by developing a comprehensive *Course Demand Estimation System*. Traditional methods of course scheduling often rely on static historical trends or manual estimations, leading to operational inefficiencies such as overcrowded classes, underutilized resources, or scheduling conflicts. The proposed system offers a data-driven solution that dynamically calculates student demand based on individual academic histories and strict curriculum logic.

The project successfully implemented an end-to-end pipeline, encompassing four key components:

- **Database Design:** A robust relational schema was adopted to model complex academic entities, including programs, pathways, prerequisite networks, and student records.
- **Synthetic Data Generation:** A sophisticated Python-based simulation engine was developed to generate a realistic experimental dataset (approx. 1,200 active students). This engine reflects diverse student behaviors (e.g., fast-track, delayed graduation) and adheres to academic constraints such as credit limits and prerequisite chains.
- **Demand Classification Logic:** Advanced SQL algorithms were implemented to categorize student demand into four actionable groups: *Retake* (High Priority), *Standard* (Curriculum-based), *Late* (Backlog clearance), and *Fast* (Advanced placement).
- **Visualization:** An interactive **Microsoft Power BI** dashboard was deployed to provide academic planners with real-time insights, allowing for "drill-down" analysis from the faculty level down to individual student cohorts.

7.2 Future Work

While the current system has successfully established a robust foundation for identifying immediate course demand, the academic planning process requires a vision that extends beyond the upcoming semester. To elevate the system from a "tactical reporting tool" to a "strategic forecasting engine," future research and development will focus on the following key advancements:

7.2.1 Multi-Horizon Forecasting using Machine Learning

Currently, the system predicts demand for the immediate next semester ($t + 1$) based on the deterministic status of the current semester (t). A significant limitation is the

inability to accurately foresee demand for semesters $t + 2$ or $t + 3$ (e.g., predicting next year’s demand).

To address this, future work will integrate **Machine Learning (ML)** models to simulate future student outcomes, enabling long-term trajectory prediction:

- **Grade Prediction Models:** We plan to train supervised learning models (e.g., *Random Forest* or *XGBoost*) using historical transcript data to predict the *probability of passing or failing* a course for each student before they even take the exam.
- **Simulation of Future States:** By applying these predicted outcomes to the current cohort, we can simulate the hypothetical academic status of students in future semesters. For instance, if the model predicts that 30% of students will fail *Data Structures* in Semester 1, the system can automatically forecast a surge in "Retake Demand" for Semester 2 and a delayed "Standard Demand" for *Algorithms* in Semester 3.
- **Strategic Value:** This allows the university to plan human resources (hiring lecturers) and facilities (building classrooms) up to **12-18 months in advance**, rather than reacting to demand only a few months prior to registration.

7.2.2 Real-time Event-Driven Architecture

Although the current dashboard offers near real-time updates when the database is queried, it still relies on a request-response model. To achieve true real-time responsiveness, specifically during critical registration periods, the architecture will be upgraded to an **Event-Driven model**.

- **Change Data Capture (CDC):** We aim to implement CDC mechanisms (using tools like *Debezium* or MySQL Binlog triggers). This ensures that any modification in the student database (e.g., a grade update, a new student enrollment, or a curriculum adjustment) immediately triggers a recalculation event.
- **Instant Dashboard Refreshes:** These events will push updates directly to the Power BI dashboard via streaming datasets. This capability is crucial during the "Add/Drop" period, allowing administrators to see the fluctuation in demand second-by-second and open new class sections instantly to resolve bottlenecks.

7.2.3 Supply-Side Optimization via Linear Programming

Currently, the system focuses solely on the demand side (how many seats are needed). Future iterations will integrate **Operations Research** techniques to solve the supply side.

- By feeding the forecasted demand into a **Linear Programming (LP)** solver (e.g., *Gurobi* or *PuLP*), the system could automatically generate an optimal class schedule that maximizes room utilization and minimizes timetable conflicts, considering constraints such as lecturer availability and room capacity.

7.2.4 Student Advisory Recommender System

Finally, the logic developed for this thesis can be repurposed to serve students directly.

- A "Personalized Roadmap" portal can be developed, where students log in to see their specific predicted path. The system would recommend exactly which courses to register for to maximize their graduation speed (based on the "Standard/Fast" logic) or recovery path (based on the "Retake" logic), acting as an automated academic advisor available 24/7.

7.3 Key Contributions

The primary contributions of this research are as follows:

1. **Logic-Based Forecasting:** Unlike simple trend analysis (Time-Series), this system employs a "bottom-up" approach, aggregating demand from individual student statuses. This ensures high accuracy by accounting for specific edge cases, such as prerequisite failures or delays caused by English proficiency requirements.
2. **Privacy-Preserving Methodology:** The successful implementation of a synthetic data generator demonstrates a viable method for developing and testing academic information systems without compromising sensitive Personally Identifiable Information (PII) of real students.
3. **Operational Efficiency:** The automation of demand estimation significantly reduces the manual workload for the Academic Affairs Office and minimizes human error in scheduling. The ability to distinguish between "Retake" and "Standard" demand empowers the faculty to allocate resources more effectively—for instance, by opening dedicated remedial classes for students with academic debts.

7.4 Concluding Remarks

In the era of Education 4.0, the digital transformation of higher education is no longer an option but a strategic imperative. As universities transition from rigid cohort-based models to flexible credit-based systems, the complexity of academic planning increases exponentially. This thesis demonstrates that the transition from intuition-based management to data-driven decision-making is not only possible but essential for modern university administration. The *Course Demand Estimation System* presented in this study serves as a robust proof-of-concept, illustrating how the rigorous application of database technology and business intelligence can solve the "chaos" of dynamic student enrollment.

The core philosophical shift proposed by this research is the transition from a *Reactive* to a *Proactive* planning paradigm. By shifting the focus from "What happened last year?" (historical heuristics) to "What do current students need?" (real-time state analysis), the system eliminates the guesswork inherent in traditional scheduling. It provides a transparent, responsive, and student-centric approach that respects the unique academic journey of every individual. Although the experimental validation was conducted on a synthetic dataset, the underlying logic is deeply grounded in the real-world academic regulations of the International University—specifically addressing

complex constraints such as prerequisite chains, English proficiency barriers, and credit limits.

Furthermore, the implications of this system extend beyond mere administrative efficiency.

- **For the University:** It represents a move towards resource sustainability. By accurately predicting demand, the university can optimize the allocation of scarce resources—classrooms, laboratories, and lecturers—thereby avoiding the financial waste of under-enrolled sections or the crisis of overcrowding.
- **For the Students:** The system acts as a silent academic advisor. By ensuring that critical "bottleneck" courses are opened with sufficient capacity, the university actively mitigates the risk of delayed graduation. This directly improves student satisfaction and supports the institution's commitment to academic success.

Ultimately, this thesis argues that data is the most underutilized asset in academic administration. The successful deployment of the *Course Demand Estimation System* creates a "Data Culture" where decisions are evidence-based rather than precedent-based. While future work is required to integrate machine learning and automate supply-side scheduling, the foundation laid by this research confirms that technology is the key to balancing operational excellence with the educational mission. By ensuring that the right courses are opened for the right students at the right time, we do not just build a better timetable; we build a smoother path to graduation for thousands of future engineers and scientists.

References

- [1] S. B. Kotsiantis, *A decision support system for forecasting students' grades*, Artificial Intelligence Review, vol. 37, no. 4, pp. 331–344, 2012.
- [2] A. Elbadrawy and G. Karypis, *Domain-aware grade prediction and top-n course recommendation*, in Proc. of the 10th ACM Conference on Recommender Systems, pp. 183–190, 2016.
- [3] A. Polyzou and G. Karypis, *Grade prediction with course and student specific models*, in Proc. of the 20th Pacific-Asia Conference on Knowledge Discovery and Data Mining, pp. 89–100, 2016.
- [4] M. Pechenizkiy, N. Trcka, P. De Bra, and P. Toledo, *Process mining in education*, Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery, vol. 4, no. 1, pp. 74–88, 2014.
- [5] G. Aldrich, et al., *Curriculum analytics: A systematic literature review*, in Proc. of the 10th International Conference on Learning Analytics & Knowledge, pp. 120–129, 2020.
- [6] D. J. Power, *Decision support systems: A historical overview*, Handbook on Decision Support Systems 1, Springer, pp. 121–140, 2008.
- [7] R. Asif, A. Merceron, S. A. Ali, and N. G. Haider, *Analyzing undergraduate students' performance using educational data mining*, Computers & Education, vol. 113, pp. 177–194, 2017.
- [8] A. Elbadrawy and G. Karypis, *Domain-aware grade prediction and top-n course recommendation*, in Proc. of the 10th ACM Conference on Recommender Systems, pp. 183–190, 2016.
- [9] S. B. Kotsiantis, *Use of machine learning techniques for educational proposals: A decision support system for forecasting students' grades*, Artificial Intelligence Review, vol. 37, no. 4, pp. 331–344, 2012.
- [10] G. Aldrich, et al., *Curriculum analytics: A systematic literature review*, in Proc. of the 10th International Conference on Learning Analytics & Knowledge, pp. 120–129, 2020.
- [11] D. J. Power, *Decision support systems: A historical overview*, Handbook on Decision Support Systems 1, Springer, pp. 121–140, 2008.
- [12] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2016.
- [13] C. Romero and S. Ventura, *Educational Data Mining: A Review of the State of the Art*, in IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews), vol. 40, no. 6, pp. 601–618, 2010.

- [14] R. S. Baker and K. Yacef, *The State of Educational Data Mining in 2009: A Review and Future Visions*, in Journal of Educational Data Mining, vol. 1, no. 1, pp. 3–17, 2009.
- [15] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed., Wiley, 2013.
- [16] M. Russo and A. Ferrari, *The Definitive Guide to DAX: Business Intelligence for Microsoft Power BI, SQL Server Analysis Services, and Excel*, 2nd ed., Microsoft Press, 2019.
- [17] A. Elbadrawy and G. Karypis, *Domain-aware grade prediction and top-n course recommendation*, in Proc. of the 10th ACM Conference on Recommender Systems, pp. 183–190, 2016.
- bibitemburke2004 E. K. Burke and S. Petrovic, *Recent research directions in automated timetabling*, in European Journal of Operational Research, vol. 140, no. 2, pp. 266–280, 2002.
- [18] N. Patki, R. Wedge, and K. Veeramachaneni, *The Synthetic Data Vault*, in Proc. of the IEEE International Conference on Data Science and Advanced Analytics (DSAA), pp. 399–410, 2016.
- [19] A. Polyzou and G. Karypis, *Grade Prediction with Course and Student Specific Models*, in Proc. of the 20th Pacific-Asia Conference on Knowledge Discovery and Data Mining (PAKDD), pp. 89–101, 2016.
- [20] S. Few, *Information Dashboard Design: The Effective Visual Communication of Data*, O’Reilly Media, 2006.
- [21] S. Finnie and P. Meng, *PhD students’ English language proficiency and their academic progress*, in Journal of Higher Education Policy and Management, vol. 27, no. 3, pp. 419–431, 2005.